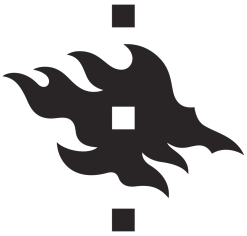
ETH zürich



UNIVERSITY OF HELSINKI

Electronic structure with general atomic orbitals on GPGPUs

Master thesis in Computational Sciences and Engineering
submitted by

Bob Schreiner
Student number: 20-944-450

supervised by:
Prof. Dr. Markus Reiher
scientific advisor:
Dr. Susi Lehtola

July 29, 2026

Abstract

Atomic orbitals have formed the backbone of computational quantum chemistry programs for the past century. While most programs rely on Gaussian-type orbitals (GTOs), arguments have been made for more general atomic orbital basis sets. The main drawback is that numerical quadrature is necessary for the integration of basis sets that do not allow for factorization. Due to the lack of computational resources, the exploration of basis sets has been limited almost exclusively to GTOs. The shift of the computational landscape towards ever more high-performance computing clusters, and the recent popularity of using general-purpose graphics processing units (GPGPUs) for scientific computations, have reopened the discussion of general atomic orbitals. With this work we provide the foundation for a reusable library, capable of harnessing the computational power of GPGPUs through the Kokkos ecosystem. The library assembles Fock and Kohn-Sham matrices for general atomic orbitals. The two-electron terms are evaluated via density fitting, and the exchange-correlation terms of the local density approximation (LDA) and the generalized gradient approximation (GGA) by quadrature on grids built with Becke's fuzzy Voronoi partitioning scheme. We show that our quadrature schemes deliver SCF energies converged to the μHa level, and we benchmark the library on the W4-11 test set using Slater-type orbitals: the mean absolute deviation of the B3LYP atomization energies improves systematically with basis size and settles at the intrinsic accuracy of the functional. We also lay out a roadmap towards larger systems.

Contents

1 Introduction

In 1929 John Lennard-Jones formalized the idea to use a linear combination of atomic orbitals (LCAO) to construct molecular orbitals (MO).[1] Twenty years later, in 1951, Clemens Roothaan used LCAO as a technique to solve the Hartree-Fock equations computationally for a general molecule.[2] From this point onward atomic orbital basis sets have formed the backbone of modern computational quantum chemistry programs. The computational cost of programs based on atomic orbitals has since been dominated by the integration of those atomic orbitals. Especially the two-electron terms in the assembly of the Fock matrix posed a major challenge to the early development of quantum chemical programs. It was due to the need for computational efficiency that the computational chemistry community almost exclusively used Gaussian-type orbitals (GTOs). The Gaussian product theorem[3] allows for analytic integration of the two-electron terms and a massive academic effort has been poured into their efficient evaluation.

The Hohenberg-Kohn theorems were published in 1964,[4] followed shortly by Kohn-Sham density functional theory (DFT).[5] The assembly of the Kohn-Sham matrix requires the integration of a non-linear functional based on the electron density. Given the complexity of the integrand the computational chemistry community was again in need of computationally efficient schemes that would allow for the further development of functionals and their integration. In 1988 Axel Becke proposed his scheme of fuzzy Voronoi partitioning[6] of the integration domain. The partitioning yielded computationally tractable quadrature schemes that overcame the limitations of earlier approaches to quantum mechanical calculations.

Curiously, Becke's scheme not only allowed for the efficient integration of exchange-correlation functionals, but in combination with a technique called density-fitting it also delivered a new method, based on quadratures, to evaluate the two-electron terms in the Fock assembly.[7,8] While programs that did not use GTOs as their underlying basis set, such as ADF,[9,10] adapted Becke's partitioning scheme to evaluate the two-electron terms, the vast majority of programs used far more efficient recurrence relations to evaluate those integrals with GTOs.[11,12]

In recent years the computational landscape has started to change. Scientific computation has shifted away from personal computers and towards high-performance computing clusters.[13] With the recent rise of general-purpose graphics processing units (GPGPU) and the advent of the exascale computing era, computational codes that were once regarded as state-of-the-art are dwarfed by other programs that can harness the computational power of GPGPUs.[14–16] In light of this paradigm shift, computations that were unthinkable thirty years ago are now within reach.

Much functionality is duplicated across the computational chemistry programs in use today, partly for reasons of optimization, but also because the source code of many industry-grade programs is not freely available. In 2023 Susi Lehtola published a paper called *A call to arms: Making the case for more reusable libraries*,[17] in which they argue that open source reusable libraries are necessary to avoid aging and unmaintainable code bases. Open source codes help researchers reproduce literature results, and they allow the community to grow and optimize libraries collectively.

With this work we lay the foundation of a reusable open source library for the evaluation of the one- and two-electron integrals constituting the Fock matrix. We also provide the infrastructure to compute exchange-correlation integrals based on the local density approximation (LDA) and the generalized gradient approximation (GGA) necessary to build Kohn-Sham matrices. Our aim is to make the library as reusable as possible across different basis sets and hardware providers. We especially target algorithms that are efficient on modern GPGPUs, but which can also run on local CPUs with lower accuracy.

This work is organized as follows. Section 2 includes the necessary theory to understand how Fock

and Kohn-Sham matrices are assembled as well as reviewing atomic orbital basis sets. Section 3 reviews the computational methods that enable us to turn theory into computation, notably Becke's fuzzy Voronoi partitioning, quadrature schemes and density fitting. Section 4 provides a detailed explanation of the most crucial implementations and GPU kernels provided by this work. Section 5 shows the convergence properties of our implementation as well as how they perform on the W4-11[18] dataset in terms of accuracy and wall time. Section 6 briefly summarizes this work's results and contributions. Section 7 gives an outlook on further research opportunities building on this work.

2 Theory

We will assume a non-relativistic framework, in which the Schrödinger equation governs the dynamics of the system. A system consists of N_{nuc} nuclei, each of which has a charge Z_I, and N_{el} electrons, each of which has charge -1. Furthermore, we apply the Born-Oppenheimer approximation, which lets us separate the Schrödinger equation into an electronic and a nuclear part. In the remainder we will focus only on the electronic Schrödinger equation and assume that the nuclei are point-like particles, fixed in space. Their coordinates are given by the set $\{\mathbf{R}_1, \cdots, \mathbf{R}_{N_{\text{nuc}}}\}$.

The electronic Schrödinger equation is then given by

$$\hat{\mathbf{H}}\mathbf{\Psi}(\mathbf{r}_1, \mathbf{r}_2, \cdots, \mathbf{r}_{N_{\text{el}}}) = E\mathbf{\Psi}(\mathbf{r}_1, \mathbf{r}_2, \cdots, \mathbf{r}_{N_{\text{el}}}) \tag{1}$$

$\hat{\mathbf{H}}$ denotes the electronic Hamiltonian, $\mathbf{\Psi}(\mathbf{r}_1, \mathbf{r}_2, \cdots, \mathbf{r}_{N_{\text{el}}})$ the many-electron wave function, E the electronic energy, and the set $\{\mathbf{r}_1, \cdots, \mathbf{r}_{N_{\text{el}}}\}$ the coordinates of the electrons.

2.1 Hartree-Fock Theory

Following the procedure of Roothaan and Hall[2,19] to solve the Hartree-Fock equations, we approximate the electronic wave function Ψ with the anti-symmetrized product of one-electron functions φ, called molecular orbitals (MO)

$$\Psi(\mathbf{r}_1, \mathbf{r}_2, \cdots, \mathbf{r}_{N_{\text{el}}}) = \frac{1}{\sqrt{N_{\text{el}}!}} \det[\varphi_1 \varphi_2 \cdots \varphi_{N_{\text{el}}}]. \tag{2}$$

We further approximate the molecular orbitals of spin $\sigma \in \{\uparrow, \downarrow\}$ by a linear combination of atomic orbitals χ

$$\varphi_i^\sigma(\mathbf{r}) = \sum_{\mu=1}^{N_{\text{bf}}} C_{\mu i}^\sigma \chi_\mu(\mathbf{r}), \tag{3}$$

and obtain a Generalized Eigenvalue Problem for each value of σ

$$\mathbf{F}^\sigma \mathbf{C}^\sigma = \mathbf{S}\mathbf{C}^\sigma \epsilon^\sigma, \tag{4}$$

where $\mathbf{F}$ is the Fock matrix, $\mathbf{C}$ holds the MO coefficients, $\mathbf{S}$ is the overlap matrix and ϵ is a diagonal matrix holding the orbital energies. Since $\mathbf{F}^\sigma$ depends on $\mathbf{C}^\sigma$ the Generalized Eigenvalue problem is non-linear and does not allow for a closed form solution. The standard practice of solving this equation is through an iterative solver, updating the matrix $\mathbf{C}^\sigma$ until self-consistency is reached. The design of self-consistent field (SCF) solvers is an ongoing research topic, which is outside of the scope of this work, since we rely on available reusable implementations.

The Fock matrix can be decomposed into one and two-electron components

$$F_{\mu\nu}^\sigma = T_{\mu\nu} + V_{\mu\nu}^{\text{nuc}} + J_{\mu\nu} - K_{\mu\nu}^\sigma, \tag{5}$$

where

$$T_{\mu\nu} = -\frac{1}{2} \langle \mu | \nabla^2 | \nu \rangle = \frac{1}{2} \int \nabla \chi_\mu(\mathbf{r}) \cdot \nabla \chi_\nu(\mathbf{r}) d\mathbf{r} \tag{6}$$

$$V_{\mu\nu}^{\text{nuc}} = -\langle\mu| \sum_{I=1}^{N_{\text{nuc}}} \frac{Z_I}{|\mathbf{r}-\mathbf{R}_I|} |\nu\rangle = \sum_{I=1}^{N_{\text{nuc}}} -Z_I \int \frac{\chi_\mu(\mathbf{r})\chi_\nu(\mathbf{r})}{|\mathbf{r}-\mathbf{R}_I|} d\mathbf{r} \tag{7}$$

are the one-electron contributions and

$$J_{\mu\nu} = \sum_{\lambda\tau} D_{\lambda\tau}^{\text{tot}} (\mu\nu|\lambda\tau) = \sum_{\lambda\tau} D_{\lambda\tau}^{\text{tot}} \iint \frac{\chi_\mu(\mathbf{r})\chi_\nu(\mathbf{r})\chi_\lambda(\mathbf{r}')\chi_\tau(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}' d\mathbf{r} \tag{8}$$

$$K_{\mu\nu}^{\sigma} = \sum_{\lambda\tau} D_{\lambda\tau}^{\sigma} (\mu\lambda|\nu\tau) = \sum_{\lambda\tau} D_{\lambda\tau}^{\sigma} \iint \frac{\chi_\mu(\mathbf{r})\chi_\lambda(\mathbf{r})\chi_\nu(\mathbf{r}')\chi_\tau(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}' d\mathbf{r} \tag{9}$$

are the two-electron contributions. $D_{\lambda\tau}^{\sigma}$ is the density matrix and is defined as

$$D_{\lambda\tau}^{\sigma} = \sum_{i=1}^{N_{\text{occ}}} f_i C_{\lambda i}^{\sigma} (C_{\tau i}^{\sigma})^*, \tag{10}$$

where f_i is the occupation number, with $0 \leq f_i \leq 1$, and $D_{\lambda\tau}^{\text{tot}} = D_{\lambda\tau}^{\uparrow} + D_{\lambda\tau}^{\downarrow}$.

$$S_{\mu\nu} = \langle\mu|\nu\rangle = \int \chi_\mu(\mathbf{r})\chi_\nu(\mathbf{r}) d\mathbf{r} \tag{11}$$

is the overlap between the basis functions χ_μ and χ_ν.

2.2 Density Functional Theory

Within Kohn-Sham DFT, the two-electron exchange term of (9) is replaced by an exchange-correlation (XC) energy functional $E_{xc}[\rho^\sigma]$, evaluated on the density $\rho^\sigma(\mathbf{r})$ defined by

$$\rho^\sigma(\mathbf{r}) = \sum_{\mu\nu} D_{\mu\nu}^{\sigma} \chi_\mu(\mathbf{r})\chi_\nu(\mathbf{r}). \tag{12}$$

Unlike the Coulomb and exact exchange integrals, $E_{xc}[\rho]$ is in general a nonlinear functional of the density and therefore admits no analogous two-electron integral representation.

The contribution of the XC term to the Kohn-Sham Fock matrix is

$$V_{\mu\nu}^{xc} = \int \chi_\mu(\mathbf{r})\, v_{xc}(\mathbf{r})\, \chi_\nu(\mathbf{r})\, d\mathbf{r}, \tag{13}$$

where $v_{xc}(\mathbf{r})$ is the XC potential, obtained from the functional derivative of E_{xc} with respect to the density,

$$v_{xc}(\mathbf{r}) = \frac{\delta E_{xc}[\rho^\sigma]}{\delta \rho^\sigma(\mathbf{r})}. \tag{14}$$

For hybrid functionals, E_{xc} additionally contains a fraction c_x of exact Hartree-Fock exchange,

$$E_{xc}^{\text{hyb}} = c_x E_x^{\text{HF}} + (1-c_x) E_x^{\text{DFT}} + E_c^{\text{DFT}}, \tag{15}$$

where E_x^{HF} is built from $K_{\mu\nu}^{\sigma}$.

Collecting these contributions, the Kohn-Sham matrix assembled during each SCF cycle reads

$$F_{\mu\nu}^{\sigma} = T_{\mu\nu} + V_{\mu\nu}^{\text{nuc}} + J_{\mu\nu} + V_{\mu\nu}^{xc} - c_x K_{\mu\nu}^{\sigma}, \tag{16}$$

which reduces to the Hartree-Fock matrix (5) for $c_x = 1$ and $V_{\mu\nu}^{xc} = 0$, and to a pure Kohn-Sham matrix for $c_x = 0$.

2.3 Basis Sets

Basis sets are almost as old as the field of quantum chemistry itself. In equation (3), we stated that molecular orbitals are approximated by a linear combination of atomic orbitals, but we never specified the exact nature of those atomic orbitals. This is because there does not exist one single basis set of atomic orbitals, but rather a whole zoo of atomic orbital basis set expansions. In each of the following subsections we will give an overview of one family of atomic basis set expansions.

2.3.1 Slater-Type Orbitals

Normalized Slater-type orbitals (STOs) can be defined as

$$\mathcal{S}_\mu(\mathbf{r}) = \mathcal{S}(\zeta, n, l, m; r, \theta, \phi) = \frac{(2\zeta)^{n+\frac{1}{2}}}{[(2n)!]^{\frac{1}{2}}} r^{n-1} e^{-\zeta r} Y_{lm}(\theta, \phi), \tag{17}$$

where ζ is a positive real number, n is their principal quantum number, l is their angular momentum and m is their magnetic quantum number. The restrictions on the quantum numbers are $n > l \geq 0$ and $l \geq m \geq -l$. $Y_{lm}(\theta, \phi)$ are the real spherical harmonics and are defined as

$$Y_{lm}(\theta, \phi) = \left(\frac{(2l+1)(l-|m|)!}{2\pi(l+|m|)!} \right)^{1/2} \times \begin{cases} \cos(|m|\phi) P_l^{|m|}(\cos\theta) & l \geq m > 0 \\ \frac{1}{\sqrt{2}} P_l^0(\cos\theta) & m = 0 \\ \sin(|m|\phi) P_l^{|m|}(\cos\theta) & -l \leq m < 0. \end{cases} \tag{18}$$

P_l^m are the associated Legendre functions.[7,20,21]

STOs are arguably the most straightforward guess for a basis set when doing molecular calculations in the LCAO framework, because STOs are modeled after the analytical solutions to the hydrogen atom.[22] It is only when being faced with the challenges of computing the two-electron integrals (8) and (9), that STOs prove problematic. Since the numerical evaluation of the two-electron integrals is computationally very expensive for STOs,[23,24] researchers have focused their attention on other basis sets that are not plagued by the same inefficiencies, namely Gaussian Type Orbitals.[3,25]

2.3.2 Gaussian-Type Orbitals

Normalized Gaussian-type orbitals (GTOs) can be defined as

$$\mathcal{G}_\mu(\mathbf{r}) = \mathcal{G}(\alpha, l, m; r, \theta, \phi) = \sqrt{\frac{2^{l+2}(2\alpha)^{l+\frac{3}{2}}}{(2l+1)!!\sqrt{\pi}}} r^l e^{-\alpha r^2} Y_{lm}(\theta, \phi), \tag{19}$$

where α is a positive real number, l is their angular momentum and m is their magnetic quantum number. The restrictions on the quantum numbers are $l \geq 0$ and $l \geq m \geq -l$. $Y_{lm}(\theta, \phi)$ are the real spherical harmonics and are defined in equation (18).[25]

GTOs are the most popular atom-centered basis set expansion in the computational quantum chemistry community,[25] mainly because of the Gaussian product theorem.[3] The Gaussian product theorem states that the product of two Gaussian functions is also a Gaussian function, and therefore allows for the analytical integration of the two-electron integrals (8) and (9). The Gaussian product theorem, combined with efficient recursive evaluation strategies, such as the Obara-Saika scheme, has made the evaluation of the two-electron integrals (8) and (9) tractable and efficient.[11,12]

GTOs come with their own set of drawbacks though. GTOs lack certain properties that one would expect the "true" underlying basis set should fulfill. For example, one would expect the basis functions to exhibit a cusp at the nucleus; this condition is known as the Kato-cusp condition.[26] The radial decay of Gaussian functions, which is much faster than that of STOs, is another typical drawback, which makes long-range interactions harder to capture in molecular calculations.[25]

The drawbacks of GTOs have not gone unnoticed and the literature is rich in basis sets that have been developed to lessen the side effects of GTOs. The usual compromise that is made when using a GTO

basis set is to add more GTOs to the original basis set, usually doubling or even tripling the original number of basis functions.[27,28] The speedup gained from using analytical integration over numerical quadrature has outweighed the need for more basis functions in the past.

2.3.3 Numerical Atom-Centered Orbitals

Numerical atom-centered orbitals (NAOs) can be defined as

$$\mathcal{N}_\mu(\mathbf{r}) = \mathcal{N}(n, l, m; r, \theta, \phi) = \frac{B_n(r)}{r} Y_{lm}(\theta, \phi), \tag{20}$$

where n is their principal quantum number, l is their angular momentum and m is their magnetic quantum number. The restrictions on the quantum numbers are $n > l \geq 0$ and $l \geq m \geq -l$. $Y_{lm}(\theta, \phi)$ are the real spherical harmonics and are defined in equation (18).[29]

NAOs can be considered a generalization of STOs and GTOs, since we could replace their radial part by analytical expressions and recover a particular basis that way.[30] Using an analytical expression for $B(r)$ would defy the purpose of using NAOs in the first place though, so NAOs are usually not defined analytically, but, as the name suggests, are only available numerically. Generating NAO basis sets is usually done by solving a simplified radial ODE for an individual atom, utilizing numerical methods such as Finite Element Methods (FEM) and storing the results numerically.[29,31–33] In the case of HelFEM the radial part $B_n(r)$ consists of a linear combination of Lagrange Interpolating Polynomials (LIP)[29,32] or Hermite Interpolating Polynomials (HIP), which can be tabulated and passed on to an integral evaluation routine.[33]

3 Computational Methods

In this section we will cover the theory of the computational methods we used to compute the Fock/KS matrices. The actual implementations are explained in Section 4.

3.1 Quadratures

Quadrature schemes are a tool to approximate integrals over a domain Ω. A quadrature scheme is usually represented by a set of tuples $\{(\mathbf{r}_q, w_q)\}_{q=1}^{N_{\text{quad}}}$, where $\mathbf{r}_q$ are the quadrature points and w_q are the quadrature weights. The integral can then be approximated by a weighted sum over function evaluations

$$I = \int_\Omega F(\mathbf{r}) d\mathbf{r} \approx \sum_{q=1}^{N_{\text{quad}}} F(\mathbf{r}_q) w_q \quad . \tag{21}$$

How the quadrature points and quadrature weights are generated can differ drastically from one scheme to another and there isn't a one-size-fits-all solution. It is common that a given quadrature scheme is specifically designed for a family of integrands F or a domain Ω or both.

We chose to implement Becke's quadrature scheme in our integral evaluation library, since it has stood the test of time and is the standard integral evaluation tool in most molecular quantum chemistry programs today.[6,10]

3.1.1 Becke's Molecular Partitioning Scheme

Becke's molecular partitioning scheme,[6] also known as fuzzy Voronoi partitioning, is a method in which the integral we want to compute is split into a weighted sum of sub-integrals. Each sub-integral is then evaluated in polar spherical coordinates, centered at a nucleus, by means of traditional quadrature schemes.

The goal of this scheme is to find relative weight functions p_n, one for each nucleus, such that for all $\mathbf{r}$,

$$\sum_n p_n(\mathbf{r}) = 1 \tag{22}$$

and such that $p_n(\mathbf{r})$ has value unity in the vicinity of its own nucleus, but vanishes continuously near any other nucleus.

Our globally defined integrand F can then be decomposed into single-center components $F_n(\mathbf{r})$,

$$F(\mathbf{r}) = \sum_n F_n(\mathbf{r}), \tag{23}$$

where

$$F_n(\mathbf{r}) = p_n(\mathbf{r})F(\mathbf{r}). \tag{24}$$

The integral I in equation (21) can then be rewritten as a sum over single-center integrations I_n over each of the nuclei in the system

$$I = \sum_n I_n, \tag{25}$$

where

$$I_n = \int F_n(\mathbf{r})d\mathbf{r}. \tag{26}$$

Each partial integral I_n can then be evaluated in spherical polar coordinates using standard numerical quadrature schemes.

Given a set of N_{nuc} spherical quadrature grids, each with n_{quad} points, $\{\{(\mathbf{r}_{nq}, w_{nq})\}_{q=1}^{n_{\text{quad}}}\}_{n=1}^{N_{\text{nuc}}}$, where we denote $(\mathbf{r}_{nq}, w_{nq})$ the q-th quadrature point generated by the n-th quadrature grid, we construct the partitioning weights by computing the confocal elliptic coordinate μ defined as

$$\mu_{ij} = \frac{r_i - r_j}{R_{ij}}, \tag{27}$$

where r_i denotes the scalar distance between the given quadrature point $\mathbf{r}_q$ and the i-th nucleus and R_{ij} denotes the distance between nucleus i and nucleus j. Then, we apply a polynomial smoothing function given as

$$h(\mu) = \frac{3}{2}\mu - \frac{1}{2}\mu^3 \tag{28}$$

k times, resulting in

$$f_1(\mu) = h(\mu) \tag{29}$$
$$f_k(\mu) = h(f_{k-1}(\mu)), \tag{30}$$

where f_k has a smoothly varying value in the interval $[-1, 1]$. The value of k controls the sharpness of the partition, $k = 3$ is the standard choice in most computer programs.[6,10] The result is then linearly mapped into the interval $[0, 1]$ by

$$s_k(\mu) = \frac{1}{2}[1 - f_k(\mu)]. \tag{31}$$

Finally we construct the normalized molecular partitioning weights p_n

$$P_n(\mathbf{r}) = \prod_{j \neq n} s_k(\mu_{nj}) \tag{32}$$

$$p_n(\mathbf{r}) = P_n(\mathbf{r}) / \sum_m P_m(\mathbf{r}). \tag{33}$$

After constructing the partitioning weights p_n, each quadrature weight w_{nq} gets scaled by its corresponding partitioning function

$$w'_{nq} = p_n(\mathbf{r}_{nq}) w_{nq}, \tag{34}$$

so that our final integral evaluation reads

$$I \approx \sum_{n=1}^{N_{\text{nuc}}} \sum_{q=1}^{n_{\text{quad}}} w'_{nq} F(\mathbf{r}_{nq}). \tag{35}$$

The construction of the relative weight functions is not unique though and improvements have been made over Becke's original proposal. Laqua et al. introduced a construction that is very close to the original proposal of Becke, but yields more robust results and has better screening properties.[34]

In their construction μ_{ij} in equation (27) is modified to

$$\mu_{ij}^{\text{mod}} = \frac{r_i - r_j}{\min(R_{\text{cutoff}}, R_{ij})}, \tag{36}$$

$$\mu_{ij}^{\text{mod,cutoff}} = \begin{cases} -1, & \mu_{ij}^{\text{mod}} \leq -1 \\ 1, & \mu_{ij}^{\text{mod}} \geq 1 \\ \mu_{ij}^{\text{mod}}, & \text{otherwise} \end{cases} \tag{37}$$

and $\mu_{ij}^{\text{mod,cutoff}}$ is used in all subsequent computations instead of μ_{ij}. $R_{\text{cutoff}} = 5$ bohr provided optimal results in their tests.

3.1.2 Quadrature Grids

Now that we know how to generate the partitioning weights p_n from a set of quadrature grids $\{\{(\mathbf{r}_{nq}, w_{nq})\}_{q=1}^{n_{\text{quad}}}\}_{n=1}^{N_{\text{nuc}}}$, we examine how exactly the quadrature grids are generated. Assuming that we are given the position of the center $\mathbf{r}_n$, in our case this would be the position of a nucleus, we can rewrite the single center sub-integral (26) in spherical polar coordinates

$$I_n = \iiint F_n(r, \theta, \phi) r^2 \sin\theta dr d\theta d\phi. \tag{38}$$

Alternatively, employing a quadrature formula for direct two-dimensional integration on the surface of the unit sphere, we can rewrite the integral as

$$I_n = \iint F_n(r, \Omega) r^2 dr d\Omega, \tag{39}$$

where Ω denotes solid angle.

The standard approach is then to use separate quadrature schemes for the angular and radial components. The most commonly employed angular quadrature grids are the ones derived by Lebedev, since they integrate the spherical harmonics exactly up to a given order l, just as the Gauss-Legendre quadrature integrates polynomials up to a given order n exactly.[35,36]

The case is not so clear-cut for the radial quadrature schemes, because not only do we have to choose an appropriate quadrature scheme, but we also need to choose a radial transformation that maps the original radial quadrature points into the interval $[0, \infty)$. In the following we will have a look at the radial quadrature scheme proposed by Becke in his original work, and the arguably most used radial quadrature grid proposed by Treutler and Ahlrichs.[6,37]

Becke proposed to use the Gauss-Chebyshev quadrature points in the interval $x \in [-1, 1]$ and use the radial transformation

$$r = \xi_n \frac{1+x}{1-x}, \tag{40}$$

where ξ_n stands for half of the Bragg-Slater radius of the respective nucleus, except for hydrogen, where Becke did not apply the factor $1/2$.

Treutler and Ahlrichs later reviewed Becke's approach to the construction of the radial grid and found that the concentration of the quadrature points was not dense enough around r_n and that too many points were generated far away from the nucleus leading to a waste of quadrature points. To mitigate those problems they proposed the radial transformation

$$(\text{M3}) : r = \frac{\xi}{\ln 2} \ln \frac{a+1}{1-x} \tag{41}$$

as well as the extended parametrized version

$$(\text{M4}) : r = \frac{\xi}{\ln 2} (a+x)^\alpha \ln \frac{a+1}{1-x}, \tag{42}$$

where they stated that $\alpha = 0.6$ yielded the best results in combination with $a = 1$ and the Chebyshev quadrature of the second kind, $x \in [-1, 1]$.

They also published a table for optimized values of ξ for the case of the (M4) grid, but it should be noted that their optimizations were conducted using a Gaussian basis set (Section 2.3.2).

Note that the generation of quadrature grids is an ongoing research topic and that the literature is rich in different approaches.[6,37–45] The stability of the quadrature grids has been heavily debated in the past and the consensus in the literature is that quadrature grids need to be well converged, i.e. have enough quadrature points, in order to yield meaningful results, especially when computing energy gradients.[46–48]

3.2 Density Fitting & Resolution of the Identity

The one-electron integrals fit the form of (21) perfectly. For example, if we wish to compute $T_{\mu\nu}$ from (6), we replace $F(\mathbf{r})$ by $\nabla\chi_\mu(\mathbf{r}) \cdot \nabla\chi_\nu(\mathbf{r})$ and provide a set of quadrature grids. The procedure explained in Section 3.1 then gives us the corresponding approximation.

In order to compute the Fock matrix though, we need to find a procedure that lets us approximate the two-electron integrals defined in (8) and (9) as well. To achieve this goal we employ density fitting, also known as the resolution of the identity, to reduce the two-electron integrals to one-electron integrals of the form (21). We closely follow the procedures described by Cohen and Handy[7] for the Coulomb integral and Watson et al.[8] for the exchange integral.

Density fitting approximates the density $\rho(\mathbf{r})$ as a linear combination of the auxiliary basis $\tilde{\chi}_\alpha(\mathbf{r})$ to give the fitted density $\tilde{\rho}(\mathbf{r})$, defined as

$$\tilde{\rho}(\mathbf{r}) = \sum_\alpha d_\alpha \tilde{\chi}_\alpha(\mathbf{r}). \tag{43}$$

The procedure of Cohen and Handy relies on minimizing the Coulomb energy E_C of the difference density $\Delta\rho(\mathbf{r})$.

$$\Delta\rho(\mathbf{r}) = \rho(\mathbf{r}) - \tilde{\rho}(\mathbf{r}) \tag{44}$$

$$E_C[\Delta\rho] = \iint \frac{\Delta\rho(\mathbf{r})\Delta\rho(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}d\mathbf{r}' \tag{45}$$

The minimization yields a set of normal equations for d_α

$$d_\alpha = \sum_\beta (\alpha|\beta)^{-1} \sum_{\mu\nu} D^{\text{tot}}_{\mu\nu}(\mu\nu|\beta). \tag{46}$$

By substituting d_α back into (43), and substituting the fitted density into equations (8) and (9), we obtain the following approximations for the Coulomb and exchange integrals

$$J_{\mu\nu} \approx \tilde{J}_{\mu\nu} = \sum_{\alpha\beta\lambda\tau} D^{\text{tot}}_{\lambda\tau}(\mu\nu|\alpha)(\alpha|\beta)^{-1}(\beta|\lambda\tau) \tag{47}$$

$$K^\sigma_{\mu\nu} \approx \tilde{K}^\sigma_{\mu\nu} = \sum_{\alpha\beta\lambda\tau} D^\sigma_{\lambda\tau}(\mu\lambda|\alpha)(\alpha|\beta)^{-1}(\beta|\nu\tau) \tag{48}$$

The 3-center integrals can then be expanded as follows

$$(\mu\nu|\alpha) = \iint \frac{\chi_\mu(\mathbf{r})\chi_\nu(\mathbf{r})\tilde{\chi}_\alpha(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}'d\mathbf{r} = \int \chi_\mu(\mathbf{r})\chi_\nu(\mathbf{r})\tilde{V}_\alpha(\mathbf{r})d\mathbf{r}, \tag{49}$$

where we defined $\tilde{V}_\alpha(\mathbf{r})$ as

$$\tilde{V}_\alpha(\mathbf{r}) = \int \frac{\tilde{\chi}_\alpha(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}', \tag{50}$$

i.e. the potential of $\tilde{\chi}_\alpha$. Note that $\tilde{V}_\alpha$ is known analytically for STOs.[7]

Similarly we can rewrite $(\alpha|\beta)$ as

$$(\alpha|\beta) = \iint \frac{\tilde{\chi}_\alpha(\mathbf{r})\tilde{\chi}_\beta(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} d\mathbf{r}'d\mathbf{r} = \int \tilde{\chi}_\alpha(\mathbf{r})\tilde{V}_\beta(\mathbf{r})d\mathbf{r}. \tag{51}$$

How $(\alpha|\beta)$ is inverted is implementation dependent and will be covered in Section 4.

The integrals (49) and (51) can then be approximated by the quadrature scheme explained in the previous section.

4 Implementation Details

This section provides a detailed explanation of our implementations of the computational methods outlined in section 3. Since we aim to build a reusable library, we have to be very careful when choosing the framework, which we want to build our library upon. First and foremost we should check if a similar reusable library already exists and can be extended to our needs.

During our literature research we examined the reusable library GauXC.[16,49] GauXC is indeed a strong candidate; the library provides a broad set of functionality to create quadrature grids, evaluate Becke's partitioning scheme and integrate Gaussian basis sets with excellent scaling on GPGPUs. The main

drawback of GauXC is that its code base is split into multiple programming languages. The authors chose to write the main library interface in C++, and the GPU kernels in CUDA and HIP.

While it is standard practice for a low level library to have a hardware specific implementation, we deem this paradigm to be unmaintainable for a scientific computing library, whose aim it is to be reused across different programs. If a change is made to one kernel on the CUDA side, after having tested and optimized this kernel, it needs to be rewritten and re-optimized in HIP and vice versa. While there exist tools like hipify, which promise to take care of this task, they only work for simple kernels, where a one to one correspondence between functions exists. For complex tasks the functionalities, which CUDA and HIP provide, are drastically different and the hard truth is that in research codes a kernel will actually only be implemented in one framework and the other one will be ignored. We came to the conclusion that in order to build a maintainable and truly reusable library, we would have to take a different approach, where the implementations themselves remain hardware agnostic.

4.1 Kokkos

The Kokkos ecosystem provides a performance portable solution to write modern C++ in a hardware agnostic way. The ecosystem consists of three main parts, Kokkos Core,[50,51] Kokkos Kernels[52] and Kokkos Tools.[53] Kokkos Core provides the base functionality to write parallel algorithms in C++. During the compilation step, Kokkos can then map an algorithm onto the desired target architectures according to architecture-specific rules to achieve best performance. Kokkos Kernels is a highly optimized software library for linear algebra and graph algorithms. The library provides native Kokkos kernels as well as the option to use third party implementations. Kokkos Tools is a software interface consisting of profiling and debugging tools. The tools are very lightweight and are meant to cause minimal friction with Kokkos applications while providing valuable insights into the Kokkos kernels.

Although Kokkos sounds like it would solve all of our problems, we observed clear drawbacks during the design and testing of our library. For example, the software developer is only able to write the kernels according to Kokkos' patterns, which means the developer has less explicit control over the data management and scheduling than one would have in a language like CUDA or HIP. The Kokkos kernels also provide a broad range of linear algebra functionality, but they fall far short of supporting all BLAS and LAPACK routines. As we will show later, in Section 4.3.1 we needed to adapt an algorithm that originally made use of a Cholesky decomposition to instead use a singular value decomposition, because the Cholesky decomposition subroutine of a general symmetric positive definite matrix is not implemented in Kokkos Kernels at the time of writing.

Table 1 summarizes how the Kokkos abstractions used throughout this work map onto the underlying GPU hardware.

Table 1: Correspondence between Kokkos abstractions and GPU hardware components.

Kokkos abstraction	Hardware component
`DefaultHostExecutionSpace`	Host
`parallel_for` / `parallel_reduce` launch	Thread Processor (grid/work distributor)
Team (`TeamPolicy` league member)	One resident thread block on one SM
`TeamThreadRange` / `TeamVectorRange`	Stream Processors within an SM
`team_scratch` / `PerTeam` scratch	Shared Memory (per SM)
`team_barrier()`	Instruction Fetch/Dispatch (block-level sync)
Plain `View` (global memory)	L1 / L2 cache → FB (device DRAM)
`MemoryTraits<RandomAccess> View`	Texture Fetch unit

4.2 Grid Construction

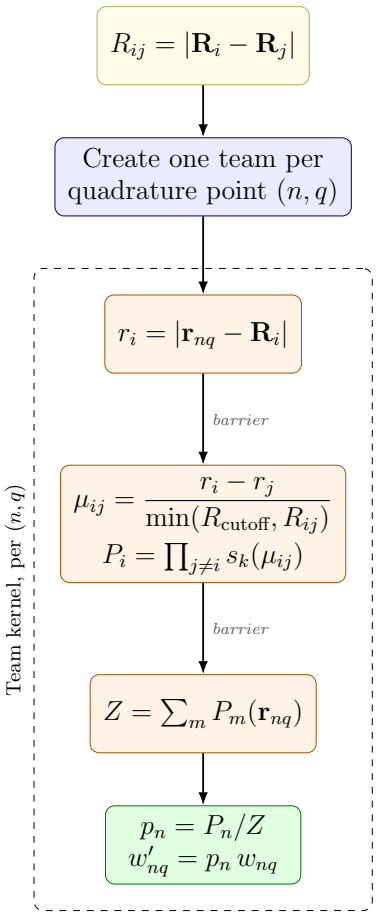
We construct the quadrature grid based on the positions of the nuclei $\{\mathbf{R}_i\}_{i=1}^{N_{\text{nuc}}}$. For each nucleus we construct a spherical quadrature grid with `IntegratorXX`,[54] and then shift these quadrature grids such that they are centered at their specific parent nucleus. We then flatten the quadrature points and weights into one dimensional `Kokkos::Views` of size $N_{\text{nuc}} \cdot n_{\text{quad}}$. Note that the number of quadrature

points generated around one nucleus could be adjusted based on its charge. This would allow for example heavier nuclei to have more quadrature points surrounding them. To keep track of which quadrature point was generated around which nucleus we create a `Kokkos::View` called *owner* of size $N_{\text{nuc}} \cdot n_{\text{quad}}$.

Now that we have fixed the position of the quadrature points, we can move on to step two of our grid construction and compute the partitioning weights. To compute Becke's partitioning scheme we make use of Kokkos' Data Parallelism paradigm as well as Kokkos' Hierarchical Parallelism paradigm. The entire control-flow is schematically represented in Figure 1 and all memory allocations are documented in Table 2.

First we compute the matrix R_{ij}, denoting the distance between nucleus i and nucleus j, and store it in a global `Kokkos::View`, accessible by all threads. Then we create one `Kokkos::Team` for each quadrature point in $\{\{(\mathbf{r}_{nq}, w_{nq})\}_{q=1}^{n_{\text{quad}}}\}_{n=1}^{N_{\text{nuc}}}$. Each team allocates two arrays of scratch memory in the L1 cache, one for the distances from the quadrature points to the nuclei r_i and one for the partitioning weights P_i. Each team then executes the following GPU kernel simultaneously.

Each team launches a `Kokkos::TeamVectorRange` parallel region, where each thread fills one entry in the r_i cache. After the team has finished filling the scratch, they proceed to calculate μ_{ij} and P_i according to Equations (27) and (33), again using a `Kokkos::TeamVectorRange` parallel region. Next the threads compute the normalization constant $Z = \sum_m P_m(\mathbf{r}_{nq})$ with a parallel reduce over all nuclei and each team computes the normalized partitioning weight $p_n(\mathbf{r}_{nq})$. Finally each team scales its quadrature point's original weight by the normalized partitioning weight, according to (34).



Allocation	Type	Size	Symbol	Cache scope
`atom_centers`	`View<Point*>`	N_{nuc}	$\mathbf{R}_i$	—
`quadrature_points`	`View<Point*>`	$N_{\text{nuc}} \cdot n_{\text{quad}}$	$\mathbf{r}_{nq}$	—
`weights`	`View<double*>`	$N_{\text{nuc}} \cdot n_{\text{quad}}$	$w_{nq} \to w'_{nq}$	—
`owner`	`View<int*>`	$N_{\text{nuc}} \cdot n_{\text{quad}}$	—	—
`R_ij`	`View<double**, RandomAccess>`	$N_{\text{nuc}} \times N_{\text{nuc}}$	R_{ij}	—
`r_cache`	`View<double*, scratch_memory_space>`	N_{nuc}	r_i	Per team
`P_cache`	`View<double*, scratch_memory_space>`	N_{nuc}	P_i	Per team

Table 2: Memory allocations in `partition_becke_team`.

Figure 1: Control-flow of the team-batched Becke partitioning routine, annotated with the underlying equations. Boxes are colored by execution scope: blue for host/serial setup, yellow for work distributed across GPU threads with data parallelism (via `Kokkos::RangePolicy`), orange for work distributed across GPU threads within a team (via `Kokkos::TeamVectorRange` or `Kokkos::TeamThreadRange`), and green for the single per-team write (via `Kokkos::single(PerTeam(...))`).

Throughout this section n_{quad} denotes the number of quadrature points generated per nucleus. The total number of quadrature points, used in the remainder of this work, is therefore $N_{\text{quad}} = n_{\text{quad}} \times N_{\text{nuc}}$.

4.3 Integral Routines

The integral routines computing the three and two center integrals of the form $(\mu\nu|\alpha)$ and $(\alpha|\beta)$ are arguably the most important implementations. For each term contributing to the Fock/Kohn-Sham matrix we implemented two different versions. The first one relies solely on dense linear algebra kernels

and is therefore perfectly suited for running on GPUs. The second one relies on locally dense linear algebra kernels and takes advantage of the local support of atom centered orbitals. Each version comes with its specific advantages and drawbacks, which we will analyze in the following subsections.

4.3.1 Dense Linear Algebra

The dense linear algebra kernel relies on the idea that GPUs are extremely good at doing dense GEneral Matrix Matrix (GEMM) and dense GEneral Matrix Vector (GEMV) multiplications. Indeed, when applying our quadrature rule given by (21) to equations (47) and (48) we can rewrite them entirely with matrix-matrix multiplications. The schemes we use to compute the Coulomb and the exchange matrix are commonly known as the RI-J and RI-K schemes.

Since $\tilde{J}_{\mu\nu}$ and $\tilde{K}_{\mu\nu}$ both depend on the density matrix $D_{\mu\nu}$, we need to recompute these matrices during each SCF cycle. Nevertheless we are able to pre-compute some quantities that can be reused within the schemes. We call $\Phi_{\mu g} = \chi_\mu(\mathbf{r}_g)$ the basis collocation matrix, $\tilde{\Phi}_{\alpha g} = \tilde{\chi}_\alpha(\mathbf{r}_g)$ the auxiliary basis collocation matrix and $\tilde{V}_{\alpha g} = \tilde{V}_\alpha(\mathbf{r}_g)$ the Coulomb collocation matrix.

Another quantity which we can pre-compute is $(\alpha|\beta)^{-1}$. We proceed by first computing the overlap matrix in the Coulomb metric

$$(\alpha|\beta) = \int \tilde{\chi}_\alpha(\mathbf{r})\tilde{V}_\beta(\mathbf{r})d\mathbf{r} \tag{52}$$

$$= \sum_g w_g \tilde{\chi}_\alpha(\mathbf{r}_g)\tilde{V}_\beta(\mathbf{r}_g) \tag{53}$$

$$= \sum_g w_g \tilde{\Phi}_{\alpha g}\tilde{V}_{\beta g} \tag{54}$$

and then use a singular value decomposition (SVD) to diagonalize the matrix $(\alpha|\beta) = USV^T = USU^T$, where U and V are orthonormal matrices and S is a diagonal matrix. Since $(\alpha|\beta)$ is a symmetric positive definite matrix we also know that $U = V$. We can then define the square-root factor $X^{\frac{1}{2}} = US^{\frac{1}{2}}$ s.t. $X^{\frac{1}{2}}(X^{\frac{1}{2}})^T = (\alpha|\beta)$. To enhance numerical stability we are also reducing the linear dependence of the auxiliary basis functions by truncating the inner dimension of the SVD to only contain the K largest singular values. Another way to think about this procedure is as a preconditioning step, where we remove all singular values below a certain threshold to lower the condition number of the inverted matrix. The final inverse then reads

$$(\alpha|\beta)^{-1} = \sum_k (X^{-\frac{1}{2}})_{\alpha k}(X^{-\frac{1}{2}})_{\beta k} \tag{55}$$

$$(X^{-\frac{1}{2}})_{\alpha k} = U_{\alpha k}(S_k)^{-\frac{1}{2}} \tag{56}$$

Another, more computationally efficient, way to compute $X^{-\frac{1}{2}}$ is with a pivoted Cholesky factorization of $(\alpha|\beta)$. We chose the SVD method because the Cholesky factorization is, as of the writing of this work, not implemented in the Kokkos Kernels library, whereas third-party library calls of the SVD to LAPACK, CUDA and HIP exist natively in Kokkos Kernels. One should also note that the diagonalization step in the startup phase is negligible compared to the overall SCF runtime.

Having precomputed all of the quantities above we are ready to tackle the SCF cycles. During each SCF cycle we provide the Fock matrix to an external SCF library, in our case we chose the reusable open source library `OpenOrbitalOptimizer`.[55] The SCF subroutine in return provides us with the density matrix $D_{\mu\nu} = \sum_i f_i C^i_\mu C^i_\nu$, where f_i are the orbital occupation numbers and C^i_μ are the molecular-orbital coefficients of the i-th occupied orbital. We then rebuild the Coulomb and exchange contributions based on the new set of orbital coefficients. For ease of notation we dropped the dependence on the spin σ, but in the case of an unrestricted calculation we get one density matrix for each possible spin value.

To avoid notation clutter we drop the upper bounds for the summation indices. In all the following equations $(\mu, \nu, \lambda, \tau) \in [1, N_{\text{bf}}]$, $(\alpha, \beta) \in [1, N_{\text{aux}}]$, $g \in [1, N_{\text{quad}}]$, $i \in [1, N_{\text{occ}}]$, $k \in [1, K]$, where N_{bf} is the number of basis functions in the standard basis set, N_{aux} is the number of basis functions in the auxiliary basis set, N_{quad} is the total number of quadrature points, N_{occ} is the number of occupied orbitals and K is the number of linearly independent auxiliary basis functions, as explained above.

The RI-J scheme essentially tries to factorize the sums and collapse the summation indices as early as possible. To see how this is done we can rearrange the terms in (47) to obtain

$$\tilde{J}_{\mu\nu} = \sum_{\alpha} (\mu\nu|\alpha) \sum_{\beta} (\alpha|\beta)^{-1} \sum_{\lambda\tau} (\beta|\lambda\tau) D_{\lambda\tau} \tag{57}$$

We can then compute a vector y that stores the intermediate coefficients. We can do this efficiently by computing the electron density ρ on every quadrature point and doing a GEMV of $\tilde{V}_{\beta g}$ with $(\rho_g w_g)$.

$$\begin{aligned}
y_\beta &= \sum_{\lambda\tau} (\beta|\lambda\tau) D^{\text{tot}}_{\lambda\tau} && (58)\\
&= \int \sum_{\lambda\tau} \chi_\lambda(\mathbf{r}) \chi_\tau(\mathbf{r}) D^{\text{tot}}_{\lambda\tau} \tilde{V}_\beta(\mathbf{r}) d\mathbf{r} && (59)\\
&= \int \rho(\mathbf{r}) \tilde{V}_\beta(\mathbf{r}) d\mathbf{r} && (60)\\
&\approx \sum_g \tilde{V}_\beta(\mathbf{r}_g) w(\mathbf{r}_g) \rho(\mathbf{r}_g) && (61)\\
&= \sum_g \tilde{V}_{\beta g} w_g \rho_g && (62)
\end{aligned}$$

We then do two applications of $X^{-\frac{1}{2}}$ via GEMV again to obtain z_α.

$$z_\alpha = \sum_\beta (\alpha|\beta)^{-1} y_\beta \approx \sum_k (X^{-\frac{1}{2}})_{\alpha k} \sum_\beta (X^{-\frac{1}{2}})_{\beta k}\ y_\beta \tag{63}$$

We compute $v_g = \sum_\alpha \tilde{V}_{\alpha g} z_\alpha$ and scale the columns of $\Phi_{\mu g}$ by $w_g v_g$. Finally we compute the result by doing a GEMM of the scaled and the unscaled collocation matrices.

$$\begin{aligned}
\tilde{J}_{\mu\nu} &= \sum_\alpha (\mu\nu|\alpha) z_\alpha && (64)\\
&= \int \chi_\mu(\mathbf{r}) \chi_\nu(\mathbf{r}) \sum_\alpha \tilde{V}_\alpha(\mathbf{r}) z_\alpha d\mathbf{r} && (65)\\
&\approx \sum_g \chi_\mu(\mathbf{r}_g) \chi_\nu(\mathbf{r}_g) w(\mathbf{r}_g) v(\mathbf{r}_g) && (66)\\
&= \sum_g \Phi_{\mu g} w_g v_g \Phi_{\nu g} && (67)
\end{aligned}$$

This scheme has an asymptotic runtime of $\mathcal{O}(N_{\text{bf}}^2 \times N_{\text{quad}})$ ($\approx \mathcal{O}(N_{\text{bf}}^3)$, if we assume that N_{quad} scales linearly with N_{bf}), where the most expensive operation is the final GEMM.

The RI-K scheme is unfortunately not as factorizable as the RI-J scheme, but we can still take advantage of the low-rank expansion of the density matrix in order to get some factorizations. The factorization then reads $(\mu i|\alpha) = \sum_\lambda C^i_\lambda (\mu\lambda|\alpha)$. The RI-K scheme then boils down to computing these 3-center integrals and applying $X^{-\frac{1}{2}}$ with a GEMM to obtain a matrix $T^i_{\mu k}$. We then perform a GEMM for each $T^i_{\mu k}$ and scale the result by f_i, before adding the term to our final matrix.

$$\tilde{K}_{\mu\nu} = \sum_i f_i \sum_{\alpha\beta\lambda\tau} C^i_\lambda (\mu\lambda|\alpha)(\alpha|\beta)^{-1}(\beta|\nu\tau)C^i_\tau \tag{68}$$

$$= \sum_i f_i \sum_{\alpha\beta} (\mu i|\alpha)(\alpha|\beta)^{-1}(\beta|\nu i) \tag{69}$$

$$\approx \sum_i f_i \sum_{\alpha\beta} (\mu i|\alpha) \sum_k (X^{-\frac{1}{2}})_{\alpha k}(X^{-\frac{1}{2}})_{\beta k}(\beta|\nu i) \tag{70}$$

$$\approx \sum_i f_i \sum_k T^i_{\mu k} T^i_{\nu k} \tag{71}$$

The explicit computation of $T^i_{\nu k}$ looks as follows

$$T^i_{\nu k} = \sum_\alpha (X^{-\frac{1}{2}})_{\alpha k} (\nu i|\alpha) \tag{72}$$

$$= \sum_\alpha (X^{-\frac{1}{2}})_{\alpha k} \int \chi_\nu(\mathbf{r}) \left(\sum_\lambda \chi_\lambda(\mathbf{r}) C^i_\lambda \right) \tilde{V}_\alpha(\mathbf{r}) d\mathbf{r} \tag{73}$$

$$= \sum_\alpha (X^{-\frac{1}{2}})_{\alpha k} \int \chi_\nu(\mathbf{r}) c^i(\mathbf{r}) \tilde{V}_\alpha(\mathbf{r}) d\mathbf{r} \tag{74}$$

$$\approx \sum_\alpha (X^{-\frac{1}{2}})_{\alpha k} \sum_g \Phi_{\nu g} w_g c^i_g \tilde{V}_{\alpha g} \tag{75}$$

The most expensive operation being again a GEMM, but compared to the RI-J scheme we need to do N_{occ} GEMMs. We end up with a total asymptotic runtime of $\mathcal{O}(N_{\text{occ}} \times N_{\text{bf}}^2 \times N_{\text{quad}}) \approx \mathcal{O}(N_{\text{bf}}^4)$.

Looking at this implementation we identify the GEMMs to be the bottleneck. Given that we are planning to run on a GPU we are in an ideal scenario since we have third-party library implementations available to us in Kokkos Kernels, which are designed to be able to take full advantage of our hardware.

The drawback of this implementation is its memory footprint. As stated above, we pre-compute and store the basis collocation matrix $\Phi_{\mu g}$, the auxiliary collocation matrix $\tilde{\Phi}_{\alpha g}$ and the Coulomb collocation matrix $\tilde{V}_{\alpha g}$. Each is a dense matrix of size $N_{\text{bf}} \times N_{\text{quad}}$ or $N_{\text{aux}} \times N_{\text{quad}}$, so the total number of stored entries is $M = (N_{\text{bf}} + 2\,N_{\text{aux}})\,N_{\text{quad}}$.

As a concrete example, we examine a moderately sized molecule with $N_{\text{nuc}} = 100$ nuclei, a quadruple-ζ Slater-type basis ($n_{\text{bf}} \approx 30$ functions per nucleus) with an auxiliary basis of $n_{\text{aux}} \approx 55$ functions per nucleus, and a medium-to-high resolution grid of $n_{\text{quad}} = 10^4$ points per nucleus. This yields $N_{\text{quad}} = 10^6$ quadrature points, so the basis collocation matrix alone holds $N_{\text{bf}} \times N_{\text{quad}} = 3 \times 10^9$ entries, i.e. 24 GB in double precision. Adding the two auxiliary matrices (5.5×10^9 entries, 44 GB each) brings the collocation data to roughly 112 GB, well beyond the memory of a single GPU. Since $\tilde{\Phi}_{\alpha g}$ is only needed to assemble $(\alpha|\beta)$ during setup, the persistent per-cycle footprint is $\Phi_{\mu g}$ and $\tilde{V}_{\alpha g}$ (≈ 68 GB).

The complete control-flow of the dense RI-J and RI-K routines, split into the one-time startup phase and the operations repeated in every SCF cycle, is summarized schematically in Figure 2.

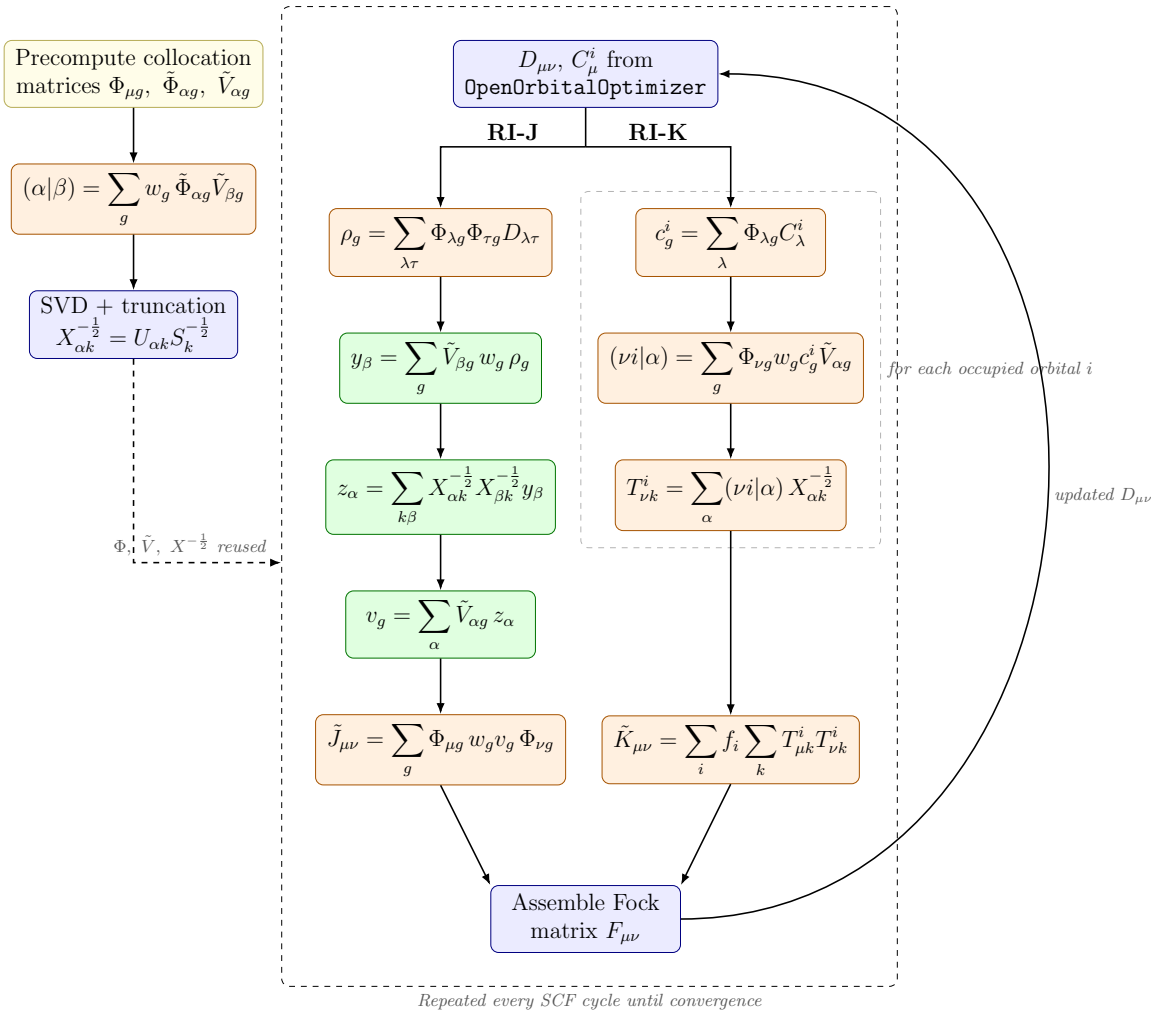


Figure 2: Control-flow of the dense RI-J and RI-K integral routines, annotated with the underlying equations. The boxes in the left-hand column form the one-time startup phase; everything inside the outer dashed region is re-evaluated in every SCF cycle, as two parallel branches that rejoin at the Fock build. Boxes are colored by operation type: blue for host orchestration and interaction with the external SCF library, yellow for pre-computed data held in device memory, orange for dense matrix–matrix products (GEMM), and green for matrix–vector products (GEMV) and element-wise scaling. The inner dashed box marks the loop over occupied orbitals in the RI-K scheme. The Fock/Kohn–Sham assembly is drawn schematically.

4.3.2 Locally Dense Linear Algebra

The locally dense linear algebra implementation of the RI-J and RI-K scheme tries to overcome the drawbacks of the dense schemes by not pre-computing the collocation matrices but instead recomputing them during each SCF cycle. Recomputing the collocation matrices allows us to take advantage of a different feature that atomic orbitals exhibit, namely their locality.

Atomic orbitals are almost always designed in a way that their radial function decays at least exponentially far away from their originating nucleus. This exponential decay lets us define a cutoff radius, after which the contributions to the integrals become negligible. Concretely, we do not need to evaluate all basis functions at every quadrature point; instead we perform a pre-screening step, where we compute a neighbor list, which matches basis functions to sets of quadrature points.

The construction of the neighbor list proceeds in the following steps. First, we create bounding boxes around points that are close to each other in space, i.e. we want to find clusters of points. Second, we compute bounding boxes around those clusters. Third, we compute a cutoff radius for each basis

function. Lastly, we compute the intersection of all spheres, generated by taking the origin of a basis function as the center and the cutoff radius as the radius, with all bounding boxes. If a sphere intersects a bounding box we add it to the neighbor list.

Fortunately there exists a reusable open source library, called `ArborX`,[56,57] written entirely in Kokkos, which we are able to seamlessly integrate into our routines. Through `ArborX`'s API we reorder our quadrature points according to a Z-order. The Z-order relies on a space filling curve, which on average makes points that are close to each other in space, also close to each other in hardware memory. We then build bounding boxes around a pre-defined number of points, i.e. a box of size N_B contains exactly N_B points. Since we previously ordered our points, we simply define one box to contain all points with indices $i \in [nN_B + 1, (n+1)N_B]$. To define the cutoff radii r_{cutoff} we use a screening threshold τ_{cutoff}, s.t. $r > r_{\text{cutoff}} \implies B(r) < \tau_{\text{cutoff}}$. Lastly, providing `ArborX` with the bounding boxes and cutoff radii, it returns the neighbor list.

In the locally dense scheme the RI-J and RI-K algorithms keep the same structure as their dense counterparts; the difference is that every operation whose inner dimension was the full set of quadrature points is replaced by a custom team-batched kernel that only visits the points and basis functions retained by the neighbor list. We launch one Kokkos team per box and, following the same hierarchical-parallelism pattern as the Becke partitioning routine (Figure 1), stage the basis-function evaluations $\chi_\mu(\mathbf{r}_g)$ through a fixed-size neighbor-tile cache in team scratch memory.

The tiled RI-J routine runs as two team kernels separated by a host-side application of $(\alpha|\beta)^{-1}$: the first accumulates the fitted-density coefficients y_α (rescaled to z_α by two GEMVs), the second contracts the fitted potential back onto basis pairs to assemble $\tilde{J}_{\mu\nu}$. Its control-flow is shown in Figure 3 and its allocations in Table 3. The tiled RI-K routine loops over the occupied orbitals: for each orbital i it folds $c^i(\mathbf{r}_g) = \sum_\lambda C^i_\lambda \chi_\lambda(\mathbf{r}_g)$ into the quadrature weights, builds the half-transformed three-center integral $(\nu i|\alpha)$ with the tiled kernel – evaluating every neighbor tile of χ_ν once and reusing it across all auxiliary functions α – and then applies $X^{-\frac{1}{2}}$ and accumulates $\tilde{K}_{\mu\nu}$ with two GEMMs. Its control-flow and allocations are given in Figure 4 and Table 4.

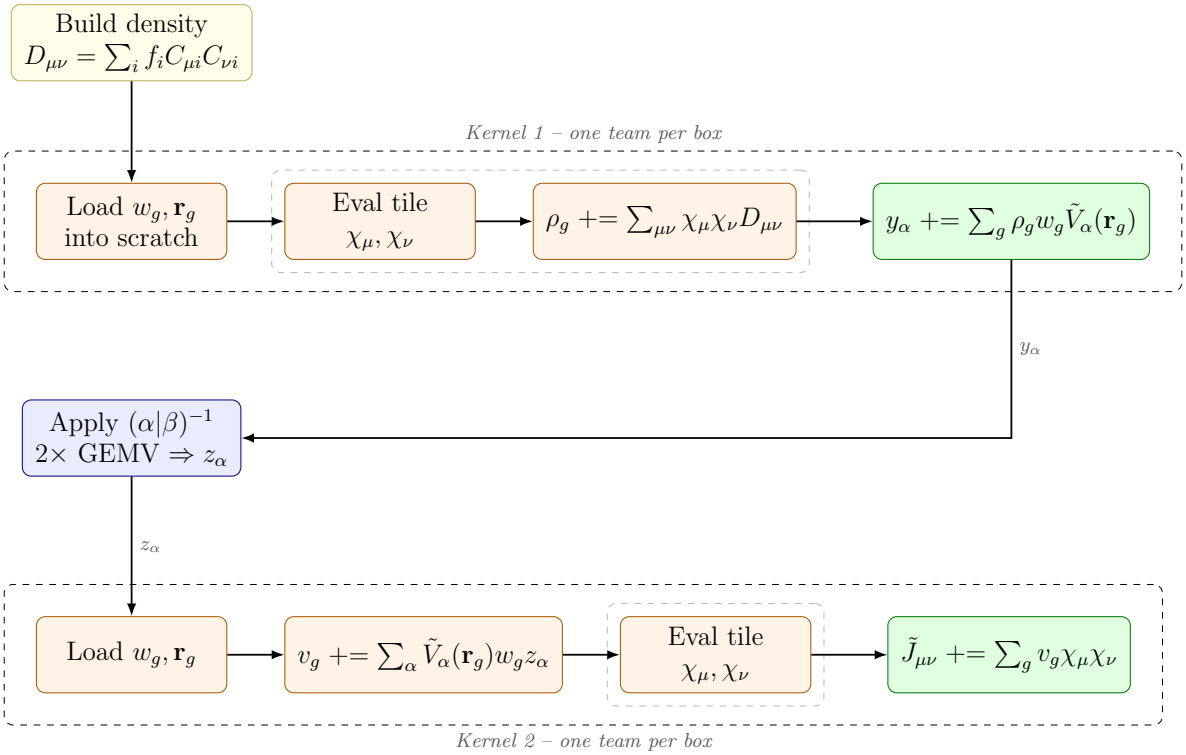


Figure 3: Control-flow of the neighbor-tiled RI-J kernel (`compute_coulomb_tiled`). One Kokkos team is launched per neighbor-list box. Kernel 1 accumulates the fitted-density coefficients y_α; a host-side application of $(\alpha|\beta)^{-1}$ (two GEMVs) rescales them to z_α; Kernel 2 assembles $\tilde{J}_{\mu\nu}$. The inner dashed boxes are the lower-triangular neighbor-tile loops, in which χ_μ, χ_ν are evaluated once into team scratch and reused. Colors follow Figure 1: yellow for data-parallel global work, orange for team-thread work in scratch, green for atomic accumulation into global memory, and blue for host / BLAS calls.

Allocation	Type	Size	Symbol	Scope
`density_matrix`	`View<double**>`	$N_{\text{bf}} \times N_{\text{bf}}$	$D_{\mu\nu}$	global
`expansion_coeff`	`View<double*>`	N_{aux}	$y_\alpha \to z_\alpha$	global
`scaling_factor`	`View<double*>`	K	—	global
`result`	`View<double**>`	$N_{\text{bf}} \times N_{\text{bf}}$	$\tilde{J}_{\mu\nu}$	global
`weights_scratch`	`View<double*, Scratch>`	$P_{\max}$	w_g	per team
`points_scratch`	`View<Point*, Scratch>`	$P_{\max}$	$\mathbf{r}_g$	per team
`{rho,pot}_scratch`	`View<double*, Scratch>`	$P_{\max}$	ρ_g / v_g	per team
`phi_a`	`View<double*, Scratch>`	`tile` $\times P_{\max}$	χ_μ tile	per team
`phi_b`	`View<double*, Scratch>`	`tile` $\times P_{\max}$	χ_ν tile	per team

Table 3: Memory allocations in `compute_coulomb_tiled`. Both kernels share the per-team scratch layout (ρ_g for Kernel 1, v_g for Kernel 2). All device views use `LayoutLeft`; $P_{\max}$ = `max_points_per_box` and `tile` = `tile_size` (default 32). The scratch footprint $(3 + 2\,\texttt{tile})\,P_{\max}$ doubles plus $P_{\max}$ points is independent of the neighbor count.

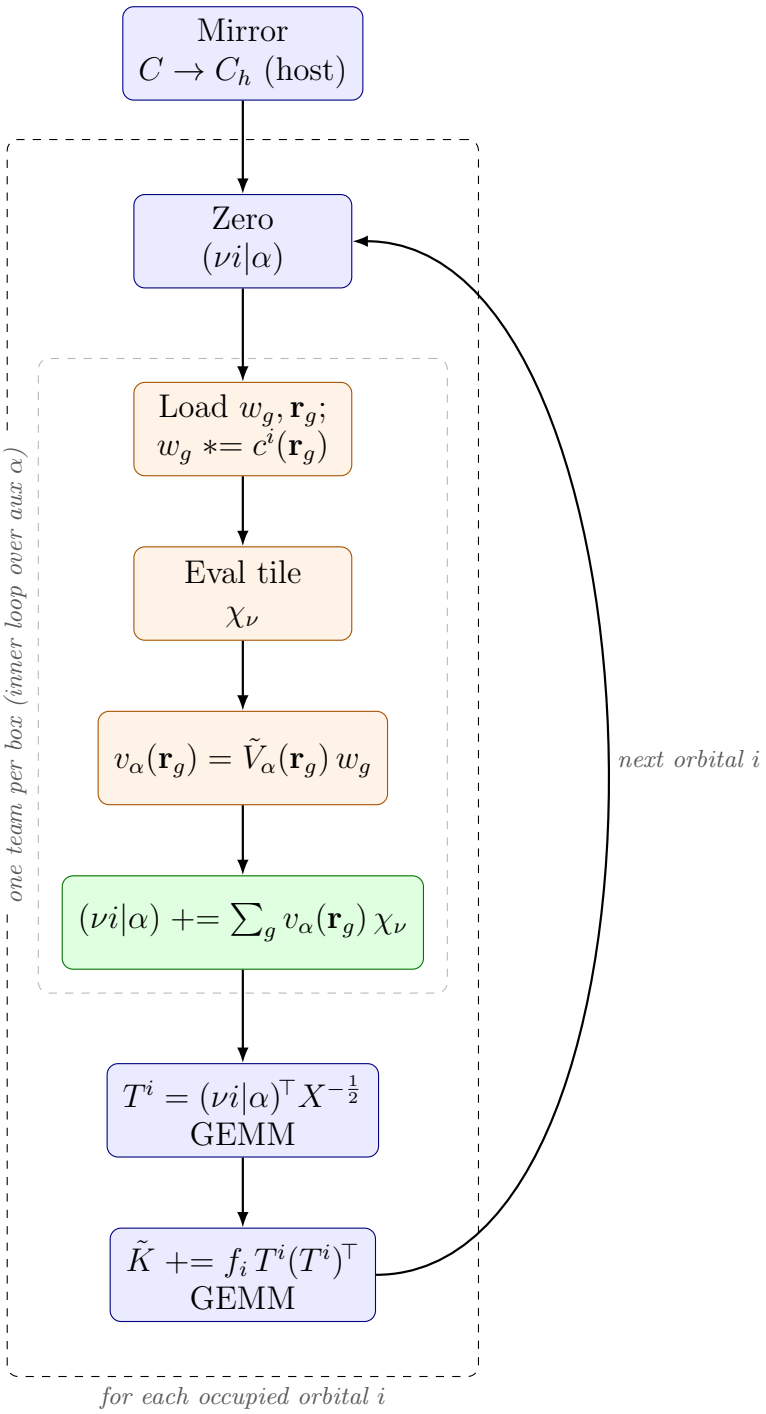


Figure 4: Control-flow of the neighbor-tiled RI-K kernel (`compute_exact_exchange_tiled`). The occupied orbitals are treated one at a time: orbital i is folded into the quadrature weights via $c^i(\mathbf{r}_g) = \sum_\lambda C^i_\lambda \chi_\lambda(\mathbf{r}_g)$, the three-center integral $(\nu i|\alpha)$ is built by the tiled team kernel (each neighbor tile of χ_ν evaluated once and reused across all auxiliary functions α), and $\tilde{K}_{\mu\nu}$ is accumulated by two host GEMMs applying $X^{-\frac{1}{2}}$. Colors as in Figure 3.

Allocation	Type	Size	Symbol	Scope
`three_center_integral`	`View<double**>`	$N_{\text{aux}} \times N_{\text{bf}}$	$(\nu i\|\alpha)$	global
`..._scaled`	`View<double**>`	$N_{\text{bf}} \times K$	$T^i_{\nu k}$	global
`result`	`View<double**>`	$N_{\text{bf}} \times N_{\text{bf}}$	$\tilde{K}_{\mu\nu}$	global
`mo_coeff_h`	host `View<double*>`	N_{occ}	f_i	host
`weights_scratch`	`View<double*, Scratch>`	$P_{\max}$	$w_g\, c^i_g$	per team
`points_scratch`	`View<Point*, Scratch>`	$P_{\max}$	$\mathbf{r}_g$	per team
`phi_cache`	`View<double*, Scratch>`	`tile` $\times P_{\max}$	χ_ν tile	per team
`potential_scratch`	`View<double*, Scratch>`	$P_{\max}$	$v_\alpha(\mathbf{r}_g)$	per team

Table 4: Memory allocations in `compute_exact_exchange_tiled`. All device views use `LayoutLeft`; $P_{\max} =$ `max_points_per_box` and `tile` $=$ `tile_size` (default 32). The three global device matrices are reused across the loop over occupied orbitals; the per-team scratch footprint $(2 +$ `tile`$)\, P_{\max}$ doubles plus $P_{\max}$ points is independent of the neighbor count.

We should note that although this implementation takes advantage of the locality that atomic orbitals provide, the repeated evaluation of the basis function becomes a massive computational cost. Another thing to keep in mind when comparing the two implementations is that due to the collocations being pre-computed in the dense linear algebra version, we could in theory also offload the pre-computation to another library, whereas in the locally dense version, we need to be able to evaluate the basis functions during the kernel.

5 Results

In this section we will present the results of our implementations. We analyze convergence as well as timing results. We will start with preliminary studies on small example systems and then later perform full benchmarks on the W4-11 set of Karton, Daon and Martin.[18]

5.1 Convergence

We first need to understand how the choice of the underlying quadrature grid affects our experiments. We therefore make an incremental study of how the resolution and geometry of a chosen quadrature grid affect the HF energies. We start with the hydrogen atom, then move on to the dihydrogen cation and lastly perform SCF computations on water. Unless an exact value is available in closed form, the energy errors reported in this section are measured against a shared self-reference, evaluated on a grid far finer than any grid in the respective sweep, rather than against an exact or basis-set-limit value. This isolates the quadrature error from the basis-set incompleteness error and lets us compare schemes on an equal footing.

5.1.1 The Hydrogen Atom

The hydrogen atom only consists of one proton and one electron. This allows us to study the convergence properties of the core Hamiltonian in isolation. Since there is no two-electron contribution, the accuracy reached here is a best case: it gives us a floor on the error we can expect for larger systems, rather than a prediction of it.

We first test how well the radial grids presented in Section 3.1 compare against each other. Figure 5 shows that all three schemes converge monotonically, with the (M4) grid being the most accurate at every radial resolution and reaching 8.3×10^{-14} Ha. The (M3) grid is the slowest of the three and still sits at 6.3×10^{-12} Ha on the finest grid of the sweep. The two finest (M4) points barely differ from one another because they have reached the noise floor of the shared reference, not because the scheme has stopped converging.

Becke's proposed radial mapping is the least accurate of the three on coarse grids—at $n_{\text{rad}} = 10$ it is more than two orders of magnitude worse than (M3)—but it overtakes (M3) from $n_{\text{rad}} = 40$ onwards and from there on sits between the (M3) and (M4) grids. That it eventually beats (M3) is somewhat surprising, since the (M3) grid was originally thought of as an improvement over Becke's proposal.

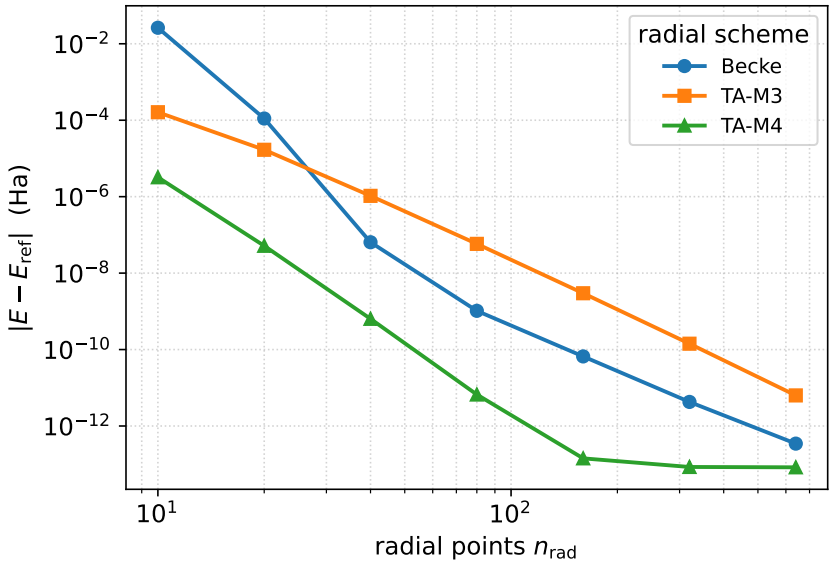


Figure 5: Radial-grid convergence of the core-Hamiltonian lowest-MO energy of a single hydrogen atom, for three radial quadrature schemes: Becke, Treutler-Ahlrichs (M3) ($\alpha = 0$) and (M4) ($\alpha = 0.6$), all with element scaling $\xi(\mathrm{H}) = 0.8$. Basis: QZ4P (ADF Slater-type). Angular grid: Lebedev, fixed at order $L = 29$. The radial point count is swept over $n_{\text{rad}} \in \{10, 20, 40, 80, 160, 320, 640\}$. The reference is the mean of the three schemes on the finest grid ($n_{\text{rad}} = 1280$), $E_{\text{ref}} \approx -0.5$ (a shared self-reference; the exact $1s$ energy is -0.5 Ha). The ordinate is $|E - E_{\text{ref}}|$.

5.1.2 The Dihydrogen Cation H_2^+

Before turning to energies, we verify the quadrature against a quantity for which the exact answer is known in closed form. For two $1s$ Slater functions with $\zeta = 1$, one centered on each proton at a separation of $R = 1$ bohr, the overlap integral is

$$S_{AB} = e^{-R}\left(1 + R + \frac{R^2}{3}\right) = \frac{7}{3}e^{-1} = 0.858385362733, \tag{76}$$

so any deviation of the numerically integrated overlap is pure quadrature error. Figure 6 shows two one-dimensional sweeps of this quantity. The radial sweep reaches an absolute error of 2.3×10^{-14} at $n_{\text{rad}} = 200$, while the angular sweep is converged to the 10^{-14} level already at $L = 23$ and merely fluctuates at that level afterwards. The atomic-centered quadrature therefore reproduces a genuine two-center integral to essentially machine precision, which tells us that the larger residual errors we observe below are a property of the integrands and not of the implementation. The data is tabulated in Table 7.

To get a better understanding of the interplay between the radial schemes and Becke's partitioning scheme, we analyze the convergence behavior of the (M4) radial mapping and Becke's original radial mapping. We compute the core Hamiltonian ground state energy of H_2^+, i.e. 2 protons and 1 electron, and perform a 2-dimensional sweep over radial and angular grid resolutions. The results of the 2-dimensional sweeps are presented in Figure 7 and are tabulated in Tables 8 and 9.

In Figure 7 we can observe that the (M4) radial mapping attains an error at least as low as Becke's for every grid combination with $n_{\text{rad}} \geq 20$, and a strictly lower one wherever the radial rather than the angular quadrature is the limiting factor. The one systematic exception is the coarsest radial grid, $n_{\text{rad}} = 10$, where Becke's mapping is the better of the two at every angular order—the same coarse-grid behavior we already observed for the hydrogen atom. We should note though that for a fixed angular order L, once the error is dominated by the angular quadrature, the error plateaus to the same value for both schemes when increasing the radial grid resolution. Hence we can only expect the (M4) radial mapping to yield superior results for a high enough angular quadrature grid. The error is moreover not monotone in L at fixed n_{rad}: at $n_{\text{rad}} = 40$ the (M4) error grows from 7.4×10^{-9} Ha at $L = 17$ to

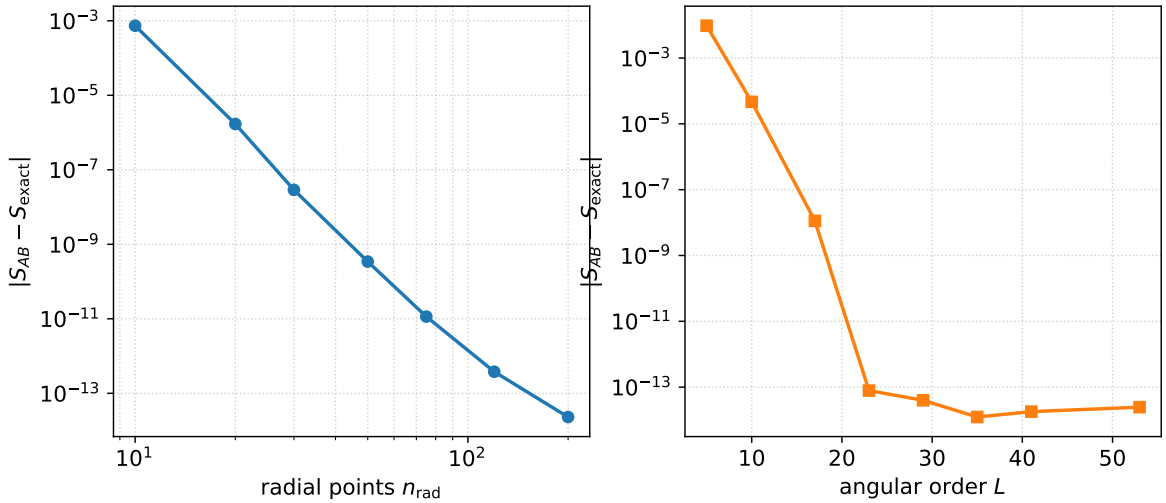


Figure 6: Convergence of the numerically integrated overlap integral S_{AB} of H_2^+ towards its exact analytic value $S_{\text{exact}} = \frac{7}{3}e^{-1} = 0.858385362733$. The system is two $1s$ Slater functions with $\zeta = 1$, one on each proton, at a separation of $R = 1$ bohr. Radial scheme: Treutler-Ahlrichs; angular scheme: Lebedev-Laikov. Left: radial sweep at a fixed angular order $L = 59$. Right: angular sweep at a fixed radial resolution $n_{\text{rad}} = 200$. Unlike the other convergence studies in this section, the reference here is exact rather than a self-reference. The full data set is listed in Table 7.

1.7×10^{-8} Ha at $L = 23$, which we attribute to a partial cancellation between the radial and angular quadrature errors that is undone as the angular grid is refined. We decided to use the (M4) grid for all further experiments, since we observed a faster convergence rate and expect the (M4) grid to be the most cost-efficient choice.

5.1.3 Self-Consistent Field Calculations

Now that we have a clear grasp on how the quadrature grids affect the convergence of simple systems, we are ready to perform the first full SCF calculations. We analyze how the unrestricted Hartree-Fock energy of water (H_2O) converges as the radial resolution and the angular order are increased together. We ran the SCF cycles on an unpruned grid, with Treutler angular grid pruning[37] and with Psi4-Robust grid pruning.[58] The grid pruning schemes are implemented in `IntegratorXX`.[54] The total HF-energies are presented in Figure 8 and the energies of the individual terms are presented in Figure 9. All energies are tabulated in Table 10.

Note that the robust pruning scheme tracks the unpruned scheme so closely that the two have the same error to three significant figures at every grid level, while using roughly 30% fewer quadrature points (965 422 instead of 1 383 454 points on the finest grid). It therefore reduces the cost of the quadrature with no discernible drawback. The Treutler pruning scheme, in contrast, seems too aggressive in its pruning, as its total error plateaus above 10^{-5} Ha.

The per-term decomposition in Figure 9 explains both this accuracy floor and the non-monotonicity of the total error. The two one-electron terms, E_{kin} and E_{ne}, converge one to two orders of magnitude more slowly than the Coulomb and exchange terms and therefore limit the overall accuracy, while the total error benefits from a partial cancellation between the individual contributions and is consequently not monotone in the grid size.

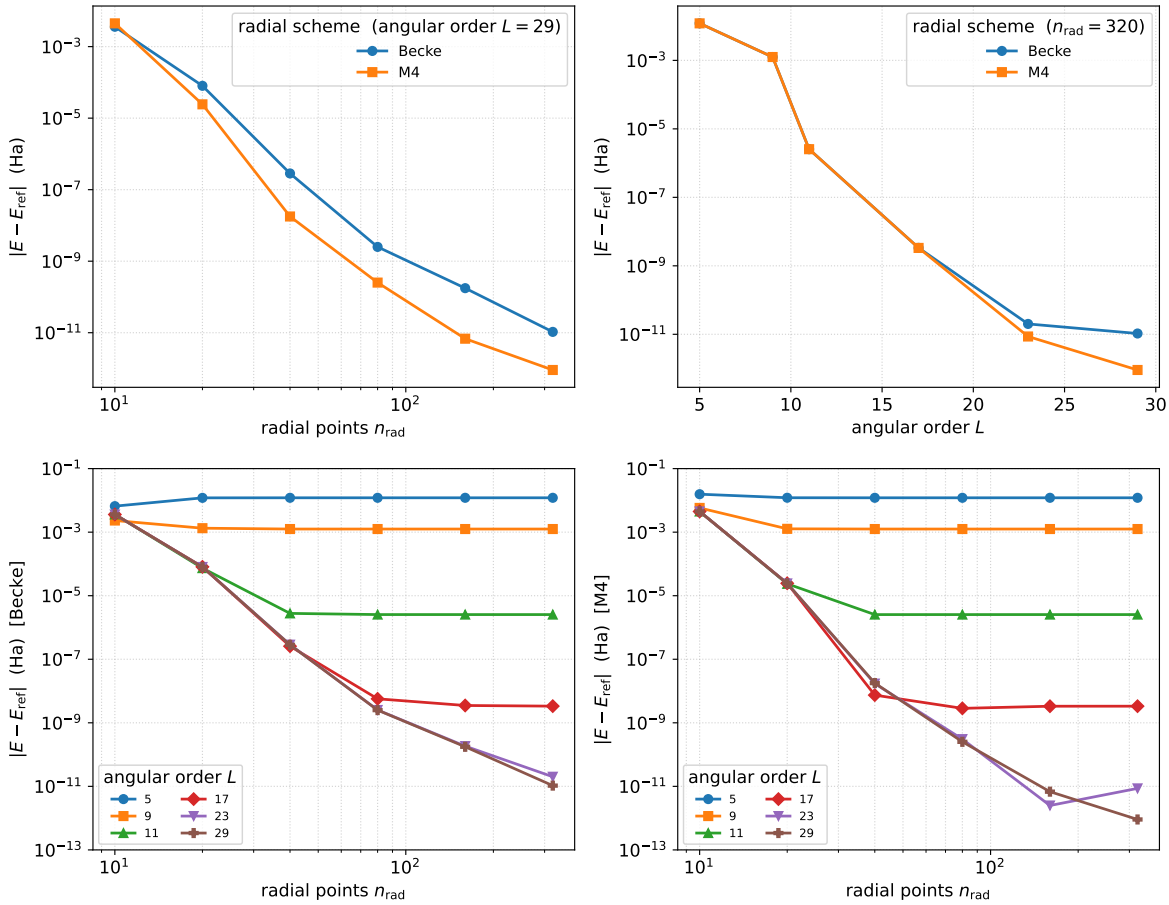


Figure 7: Two-dimensional (radial × angular) grid convergence of the H_2^+ core-Hamiltonian ground-state (lowest-MO) energy, comparing the Becke and Treutler-Ahlrichs (M4) radial schemes on a shared Lebedev angular grid. Molecule: H_2^+ with protons at $x = 0$ and $x = 1$ bohr; basis: QZ4P. Top left: radial convergence at the finest angular order in the sweep ($L = 29$). Top right: angular convergence at the finest radial grid ($n_{\mathrm{rad}} = 320$). Bottom: for each scheme, $|E - E_{\mathrm{ref}}|$ versus n_{rad} with one curve per angular order $L \in \{5, 9, 11, 17, 23, 29\}$, illustrating that each radial curve plateaus at a floor set by the angular order (and vice versa). The reference is the mean of the two schemes on the finest grid ($n_{\mathrm{rad}} = 640$, $L = 41$), $E_{\mathrm{ref}} = -1.45175161440473$ Ha (shared self-reference; the two schemes agree there to 7×10^{-13} Ha). The ordinate is $|E - E_{\mathrm{ref}}|$.

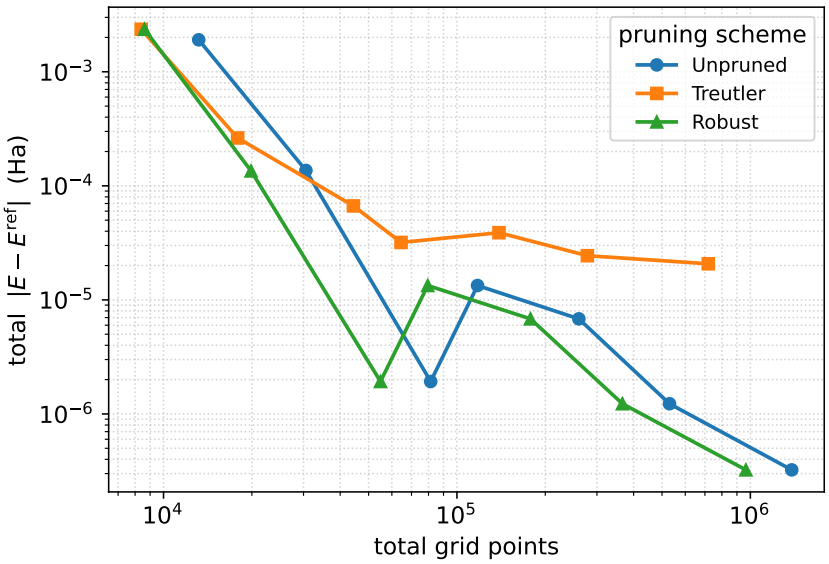


Figure 8: Angular pruning-scheme comparison on water (H_2O): convergence of the *total* unrestricted Hartree-Fock energy, plotted as accuracy per grid point. Schemes: *Unpruned* (full Lebedev order on every radial shell), *Treutler* (inner/middle shells fixed at Lebedev orders 7/11) and *Robust* (inner order 7, middle order base − 6, which tracks the base order). Radial scheme: TA-M4; angular: Lebedev; basis: QZ4P with QZ4P fit set; uniform radial sizing. Each grid level grows n_{rad} and the Lebedev order together, with $(n_{\mathrm{rad}}, L) \in \{(40, 17), (60, 21), (90, 28), (130, 29), (200, 35), (300, 41), (600, 45)\}$. The shared self-reference is a uniform unpruned grid with $n_{\mathrm{rad}} = 1000$, $L = 50$ (2 917 774 points), $E_{\mathrm{scf}} = -76.0667767173$ Ha. Note that the total error is not monotone: it results from the cancellation of the individual energy terms shown in Figure 9. The full data set is listed in Table 10.

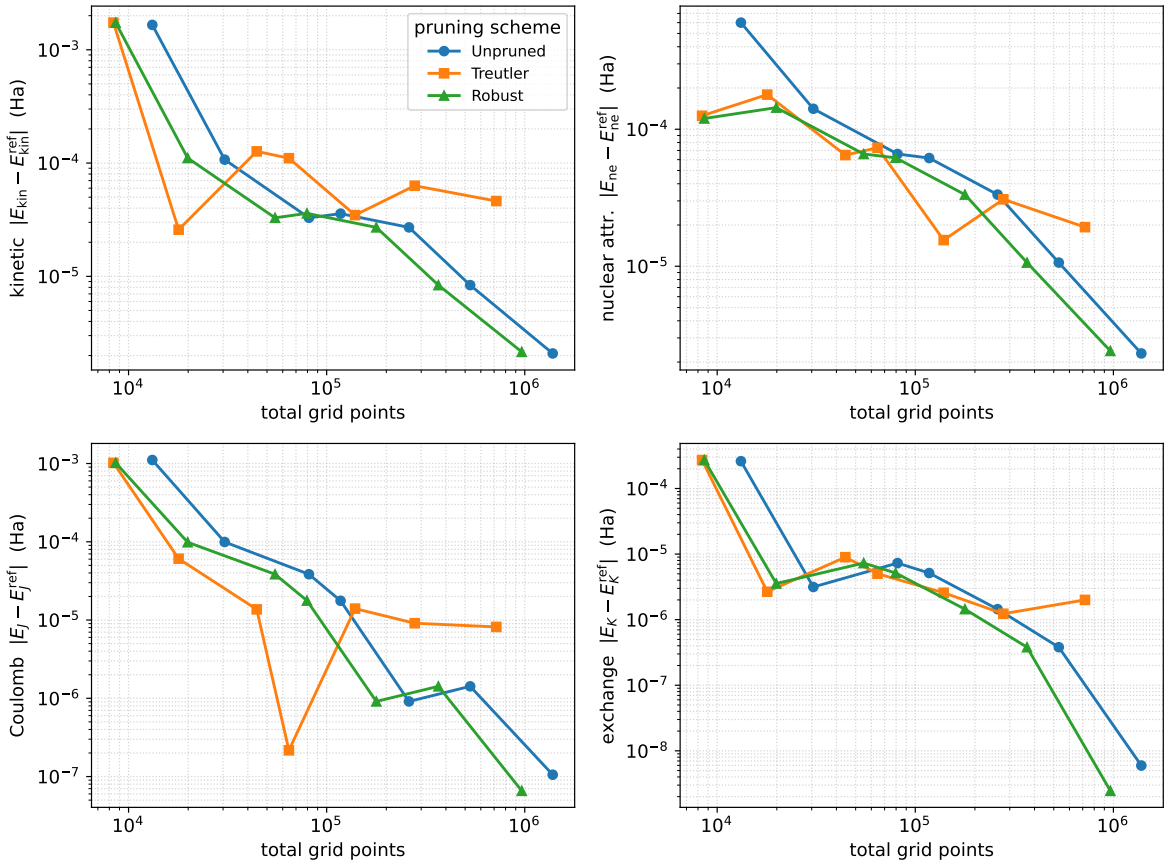


Figure 9: Per-term decomposition of the angular pruning-scheme comparison of Figure 8: absolute errors of the kinetic (E_{kin}), nuclear-attraction (E_{ne}), Coulomb (E_J) and exact-exchange (E_K) contributions to the unrestricted Hartree-Fock energy, against the total number of grid points. Systems, schemes, grids and reference as in Figure 8; the reference values are $E_{\mathrm{kin}} = 75.9976267410$, $E_{\mathrm{ne}} = -199.0472994426$, $E_J = 46.7402184304$ and $E_K = -8.9450726577$ Ha. The two one-electron terms converge one to two orders of magnitude more slowly than the two-electron terms and therefore limit the overall accuracy. The full data set is listed in Table 10.

5.2 Benchmarks

In the previous section we established that our quadrature converges to the level of a few μHa. We now want to benchmark the complete code on a realistic workload. We ask two questions. The first one being, how well we can reproduce chemical quantities. The second one being, where do our integration routines spend their time.

For both questions we use the W4-11 test set of Karton, Daon and Martin,[18] which collects 140 total atomization energies of small molecules, consisting of first and second row elements. The reference values are zero-point exclusive, non-relativistic and defined at clamped nuclei. This is exactly the quantity that a difference of converged SCF total energies gives, so no thermochemical or relativistic corrections are needed. We run the whole set with the B3LYP hybrid functional[59] in each of the four ADF Slater-type basis sets DZ, TZP, TZ2P and QZ4P. The set contains 152 distinct species, which amounts to 608 SCF calculations in total. All calculations use the Treutler-Ahlrichs (M4) radial grid with 100 radial points, a Lebedev angular grid of order $L = 35$ with the robust pruning scheme of Section 4.2, and the dense algorithm. The SCF is converged to a DIIS error of 10^{-7}, which we relax to 10^{-6} for open shell species because their partially filled degenerate shells leave a small residual gradient that DIIS cannot remove even when the energy is long converged. All timings were measured on a single GH200 GPU.

A small number of open shell species does not converge. We exclude every reaction that contains such

a species rather than report an energy the solver never accepted, which leaves 139, 131, 118 and 121 of the 140 reactions for DZ, TZP, TZ2P and QZ4P respectively.

Two caveats apply to the interpretation of the results. The quantity we measure is the sum of the basis set incompleteness error and the intrinsic error of the B3LYP functional. It is not a measure of the numerical error of the integrator, which the previous section places three to four orders of magnitude below the deviations reported here. Furthermore, W4-11 deliberately contains several strongly multireference species such as C_2 and FO_2, for which a single determinant hybrid functional is not expected to work well.

5.2.1 Energetic Accuracy

Figure 10 shows the distribution of the signed atomization energy error for each basis set, and Table 5 collects the corresponding summary statistics. The distributions narrow and shift towards zero monotonically with growing basis size. The mean absolute deviation improves from 52.9 kcal/mol at DZ to 8.9 kcal/mol at TZP, 6.2 kcal/mol at TZ2P and 5.1 kcal/mol at QZ4P. The DZ basis carries no polarization functions at all and underbinds so severely that it is of no quantitative use. Adding a single polarization shell at TZP is by far the largest single improvement, and the remaining progression to TZ2P and QZ4P is comparatively modest. The QZ4P distribution settles at a mean signed deviation of -4.0 kcal/mol rather than at zero. This residual is the expected behavior of B3LYP itself and not a deficiency of our integration, since the functional is known to underbind atomization energies. We take the agreement of this progression with the published behavior of B3LYP as evidence that the full pipeline of grid construction, density fitting and exchange-correlation quadrature works correctly on realistic chemistry.

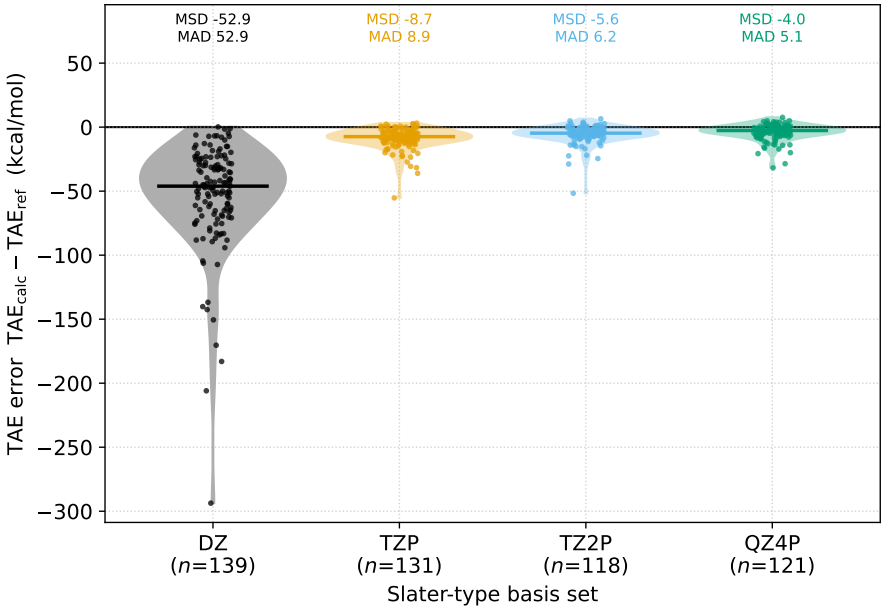


Figure 10: Distribution of the signed error in the W4-11 total atomization energies, one violin per Slater-type basis set, with the number of converged reactions given below each label. Method is unrestricted B3LYP (Libxc `hyb_gga_xc_b3lyp3`, the VWN3 parametrization) with the dense algorithm. The grid is TA-M4 radial with $n_{\mathrm{rad}} = 100$ and Lebedev order $L = 35$ with robust pruning and uniform radial sizing. The linear dependence threshold of 10^{-4} is applied uniformly to all four basis sets. Reference values are the zero-point exclusive, non-relativistic, clamped-nuclei atomization energies of Ref.[18] Individual reactions are overlaid as jittered points and the horizontal bar marks the median.T he summary statistics are given in Table 5.

Table 5: Summary of the W4-11 B3LYP atomization energy errors per basis set. Listed are the number of reactions N_{rx} for which all species converged, the mean number of primary basis functions $\overline{N_{\mathrm{bf}}}$, the mean signed deviation (MSD), the mean absolute deviation (MAD), the root mean square deviation (RMSD), the largest absolute deviation and the mean wall time per species $\bar{t}$. Systems, method and grid as in Figure 10.

Basis	N_{rx}	$\overline{N_{\mathrm{bf}}}$	MSD (kcal/mol)	MAD (kcal/mol)	RMSD (kcal/mol)	max (kcal/mol)	$\bar{t}$ (s)
DZ	139	29	-52.88	52.88	66.85	293.68	1.5
TZP	131	55	-8.72	8.94	11.91	55.26	2.1
TZ2P	118	79	-5.64	6.23	9.06	51.65	3.3
QZ4P	121	138	-4.04	5.13	7.42	31.82	9.9

5.2.2 Cost and Time Distribution

Figure 11 plots the measured wall time against the number of primary basis functions. The left panel gives the total runtime and the right panel the cost of a single Fock build. Dividing by the number of Fock builds removes the variation in how many SCF iterations each molecule happened to need, so the right panel isolates the scaling of the integral kernels themselves. The fitted exponents rise with the size of the basis, which reflects that the richer sets place proportionally more of the work in the dense linear algebra steps that scale most steeply.

We deliberately do not use the number of quadrature points on the horizontal axis. With uniform radial sizing the molecular grid contains $n_{\mathrm{atoms}} \times n_{\mathrm{rad}} \times n_{\mathrm{ang}}$ points independent of the basis set, so it takes only as many distinct values as there are molecule sizes in the set and cannot resolve the cost differences between the four bases.

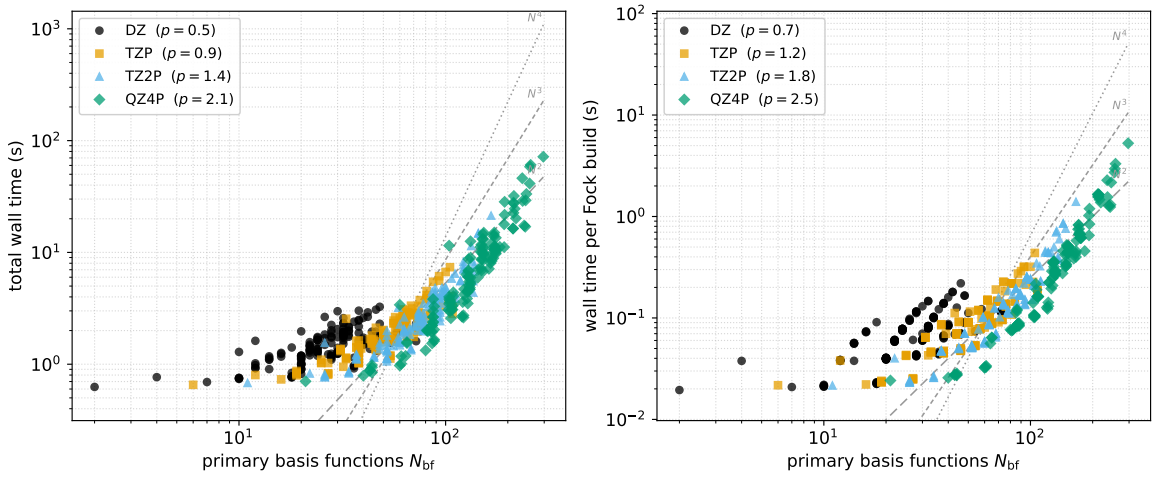


Figure 11: Wall time over the W4-11 set against the number of primary basis functions N_{bf}, on log-log axes. The left panel shows the total program runtime including grid construction, basis loading and all SCF iterations. The right panel shows the mean wall time of one Fock build, that is the total time spent in Fock construction divided by the number of builds, which removes the per molecule variation in the SCF iteration count. Legend entries give the exponent p of a least squares power law $t \propto N_{\mathrm{bf}}^{p}$ fitted to each basis set. The gray guides are N^2, N^3 and N^4 slopes anchored at the median of the data. Systems, method and grid as in Figure 10.

Figure 12 decomposes the runtime into the individual stages of the calculation, and the outcome is very clear. In the small DZ basis the exchange-correlation quadrature dominates with about three quarters of the total time. Its absolute cost however stays nearly constant across the four bases, at roughly 0.7 to 1.0 s per molecule, because it is bound by the size of the grid and not by the size of the

basis. The exact exchange build behaves in the opposite way. It grows from 0.1 s per molecule at DZ to 6.9 s at QZ4P, where it accounts for 84 percent of the total runtime and leaves grid construction, collocation, the core Hamiltonian, the auxiliary overlap of the density fitting and the Coulomb build as minor contributions. This is the expected consequence of evaluating K through the density fitted route of Section 3.2 once per spin channel in every iteration, and it identifies the exact exchange kernel as the component where optimization would pay off most.

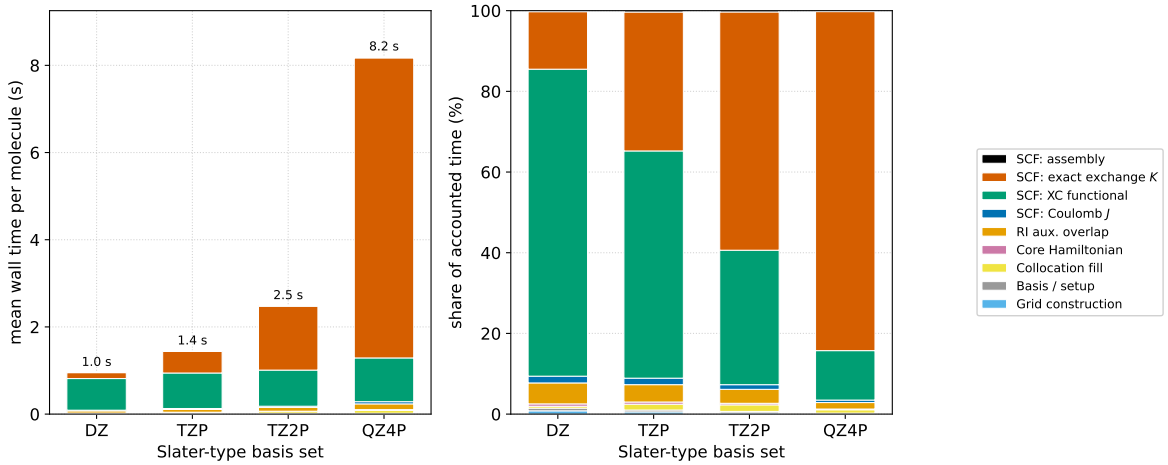


Figure 12: Distribution of wall time over the stages of the calculation, averaged over all converged W4-11 species and summed over every SCF iteration of each run. The left panel shows the absolute mean wall time per species with the total annotated above each bar. The right panel shows the same data as a percentage of the accounted time, which makes the shift of work into the exact exchange build with growing basis size explicit. The first five categories are incurred once during start-up and the four labeled *SCF* once per Fock build. Systems, method and grid as in Figure 10. All timings use the dense algorithm.

6 Conclusion

We found that the (M4) radial grid mapping has better convergence properties than the (M3) and (Becke) radial mappings. When performing Hartree-Fock SCF calculations the error converges to within a few μHa, with the one-electron terms posing the accuracy bottleneck; the two-electron terms converge to sub-μHa levels for sufficiently dense quadrature grids. Regarding pruning schemes we found that the Treutler pruning scheme plateaus above 10^{-5} Ha, whereas the Psi4-Robust pruning scheme reduces the number of quadrature points by about 30% with no discernible loss of accuracy.

Our benchmarks show that our quadratures in combination with SCF libraries reproduce the published behavior of B3LYP on the atomization energies of the W4-11 set. We observe a systematic improvement towards the complete basis set limit going from DZ $\rightarrow$ TZP $\rightarrow$ TZ2P $\rightarrow$ QZ4P, which lowers the mean absolute deviation from 52.9 to 5.1 kcal/mol and leaves a residual mean signed deviation of -4.0 kcal/mol that reflects the intrinsic underbinding of the functional rather than our integration.

The timings reveal that the cost shifts into the exact exchange build as the basis grows: it is a minor contribution at DZ, where the exchange-correlation quadrature dominates with about three quarters of the runtime, but takes up 84% of the computational time for the QZ4P basis set. With an average build time rising from 0.07 s per Fock matrix at DZ to 0.53 s at QZ4P for molecules with a handful of nuclei, the usefulness of the library is still limited to very small systems. The biggest benefit would come from optimizations in the locally dense kernels from Section 4.3.2, whose implementations are still in a premature state.

Nevertheless, the reusability of the dense implementations should not be understated. The integration kernels solely rely on a handful of precomputed quantities, such as the collocation matrix, which can be

provided by the calling program. Our library opens access to modern GPGPU computational power for Fock and Kohn-Sham matrix assembly.

7 Further Research

We believe that our library has the potential to accelerate the exploration of NAOs. We initially planned to run the benchmarks with NAOs constructed with HelFEM,[29,32] but due to time constraints we settled for STOs provided by ADF,[9] since they were readily available and we could rely on more literature during the debugging and testing of the code. We think that a study exploring the convergence properties of the NAOs on polyatomic systems is of great interest.

At present the code can only handle small systems. Extending the code with MPI would allow for distributed computing. Harnessing the power of multiple GPUs will likely make it possible to study systems substantially larger than the ones considered here. Since the most computationally expensive part is a GEMM, and the Fock assembly can be done with close to no communication overhead, we expect the parallel scaling in the number of GPUs to be near linear.

To break past the barrier of moderately sized systems and move towards extreme-scale calculations, the locally dense implementations would need substantial optimizations. Our current implementations require the basis functions to be evaluated on the fly. At the time of writing the kernels are latency-bound: they do not expose enough independent work to hide memory and instruction latency, and therefore cannot harness the full potential of GPGPUs, which benefit from high arithmetic intensity and regular dense linear algebra operations.

Another option to reduce the computational cost would be to reduce the number of required quadrature points. We already observed that robust pruning of the angular grids removes about 30% of the quadrature points at no cost in accuracy, which should translate into a comparable saving in wall time, since the computational time grows linearly with the grid size. Since the literature has been dominated by GTOs, the published quadrature grids have also been optimized for their integration. We expect that adaptive schemes that are directly connected to the convergence of the energies could yield improvements in overall computational cost.

Acknowledgments

I want to thank Prof. Dr. Markus Reiher for enabling me to conduct my thesis externally at the University of Helsinki. My thanks go to Dr. Susi Lehtola for proposing the thesis project and for their continued support and mentorship throughout the writing of this thesis. I am grateful to Philip, Milan, Hugo, Jonde, Kilian and Luukas for the moral support and positive group mentality they provided in the office. I also thank CSC – IT Center for Science, Finland, for providing the computational resources used for the experiments in this work. Lastly, I want to thank my friends and partners for their support outside of the working hours I spent on this thesis; without them I would not have been able to finish this work.

A Convergence Data

A.1 Hydrogen Atom: Radial Schemes

n_{rad}	N_{quad}	$\lvert E - E_{\mathrm{ref}}\rvert$ Becke	$\lvert E - E_{\mathrm{ref}}\rvert$ TA-M3	$\lvert E - E_{\mathrm{ref}}\rvert$ TA-M4
10	3,020	2.62×10^{-2}	1.62×10^{-4}	3.21×10^{-6}
20	6,040	1.11×10^{-4}	1.68×10^{-5}	5.18×10^{-8}
40	12,080	6.42×10^{-8}	1.05×10^{-6}	6.36×10^{-10}
80	24,160	1.03×10^{-9}	5.82×10^{-8}	6.65×10^{-12}
160	48,320	6.61×10^{-11}	2.97×10^{-9}	1.42×10^{-13}
320	96,640	4.26×10^{-12}	1.42×10^{-10}	8.48×10^{-14}
640	193,280	3.46×10^{-13}	6.31×10^{-12}	8.30×10^{-14}

Table 6: Radial-grid convergence data of the hydrogen-atom study (Figure 5). Absolute error of the core-Hamiltonian lowest-MO energy per radial scheme.

A.2 H_2^+: Overlap Integral

sweep	param.	N_{quad}	S_{AB}	$\lvert S_{AB} - S_{\mathrm{exact}}\rvert$
radial (n_{rad})	10	23,990	0.859122134805	7.37×10^{-4}
radial (n_{rad})	20	47,982	0.858383667212	1.70×10^{-6}
radial (n_{rad})	30	71,964	0.858385391682	2.89×10^{-8}
radial (n_{rad})	50	119,946	0.858385362390	3.43×10^{-10}
radial (n_{rad})	75	179,918	0.858385362722	1.15×10^{-11}
radial (n_{rad})	120	287,864	0.858385362733	3.79×10^{-13}
radial (n_{rad})	200	479,774	0.858385362733	2.31×10^{-14}
angular (L)	5	5,414	0.848794055322	9.59×10^{-3}
angular (L)	10	19,814	0.858431911821	4.65×10^{-5}
angular (L)	17	43,814	0.858385374009	1.13×10^{-8}
angular (L)	23	77,414	0.858385362733	7.84×10^{-14}
angular (L)	29	120,614	0.858385362733	3.97×10^{-14}
angular (L)	35	173,246	0.858385362733	1.23×10^{-14}
angular (L)	41	235,302	0.858385362733	1.80×10^{-14}
angular (L)	53	388,758	0.858385362733	2.45×10^{-14}

Table 7: Convergence data of the H_2^+ overlap-integral study (Figure 6); $S_{\mathrm{exact}} = \frac{7}{3}e^{-1} = 0.858385362733$.

A.3 H_2^+: Radial × Angular Sweep

$n_{\mathrm{rad}} \setminus L$	5	9	11	17	23	29
10	6.59×10^{-3}	2.32×10^{-3}	3.62×10^{-3}	3.60×10^{-3}	3.60×10^{-3}	3.60×10^{-3}
20	1.20×10^{-2}	1.33×10^{-3}	7.38×10^{-5}	8.10×10^{-5}	8.02×10^{-5}	8.02×10^{-5}
40	1.21×10^{-2}	1.25×10^{-3}	2.78×10^{-6}	2.58×10^{-7}	2.88×10^{-7}	2.84×10^{-7}
80	1.21×10^{-2}	1.25×10^{-3}	2.55×10^{-6}	5.69×10^{-9}	2.53×10^{-9}	2.51×10^{-9}
160	1.21×10^{-2}	1.25×10^{-3}	2.55×10^{-6}	3.51×10^{-9}	1.85×10^{-10}	1.76×10^{-10}
320	1.21×10^{-2}	1.25×10^{-3}	2.55×10^{-6}	3.35×10^{-9}	2.01×10^{-11}	1.06×10^{-11}

Table 8: H_2^+ two-dimensional grid-convergence data, Becke radial scheme (Figure 7): $\lvert E - E_{\mathrm{ref}}\rvert$ in Ha for every combination of n_{rad} and angular order L.

$n_{\mathrm{rad}} \setminus L$	5	9	11	17	23	29
10	1.57×10^{-2}	5.79×10^{-3}	4.48×10^{-3}	4.47×10^{-3}	4.47×10^{-3}	4.47×10^{-3}
20	1.21×10^{-2}	1.27×10^{-3}	2.37×10^{-5}	2.46×10^{-5}	2.44×10^{-5}	2.43×10^{-5}
40	1.21×10^{-2}	1.25×10^{-3}	2.54×10^{-6}	7.44×10^{-9}	1.71×10^{-8}	1.78×10^{-8}
80	1.21×10^{-2}	1.25×10^{-3}	2.55×10^{-6}	2.85×10^{-9}	3.06×10^{-10}	2.52×10^{-10}
160	1.21×10^{-2}	1.25×10^{-3}	2.55×10^{-6}	3.33×10^{-9}	2.50×10^{-12}	6.80×10^{-12}
320	1.21×10^{-2}	1.25×10^{-3}	2.55×10^{-6}	3.33×10^{-9}	8.63×10^{-12}	9.07×10^{-13}

Table 9: H_2^+ two-dimensional grid-convergence data, M4 radial scheme (Figure 7): $|E - E_{\mathrm{ref}}|$ in Ha for every combination of n_{rad} and angular order L.

A.4 Water: Angular Pruning Schemes

n_{rad}	L	N_{quad}	$\lvert\Delta E_{\mathrm{kin}}\rvert$	$\lvert\Delta E_{\mathrm{ne}}\rvert$	$\lvert\Delta E_J\rvert$	$\lvert\Delta E_K\rvert$	$\lvert\Delta E_{\mathrm{tot}}\rvert$
Unpruned							
40	17	13,180	1.66×10^{-3}	6.00×10^{-4}	1.11×10^{-3}	2.61×10^{-4}	1.91×10^{-3}
60	21	30,568	1.07×10^{-4}	1.41×10^{-4}	9.94×10^{-5}	3.15×10^{-6}	1.36×10^{-4}
90	28	81,376	3.29×10^{-5}	6.60×10^{-5}	3.85×10^{-5}	7.30×10^{-6}	1.93×10^{-6}
130	29	117,542	3.58×10^{-5}	6.16×10^{-5}	1.76×10^{-5}	5.14×10^{-6}	1.34×10^{-5}
200	35	260,274	2.71×10^{-5}	3.33×10^{-5}	9.12×10^{-7}	1.45×10^{-6}	6.82×10^{-6}
300	41	530,152	8.37×10^{-6}	1.06×10^{-5}	1.42×10^{-6}	3.80×10^{-7}	1.23×10^{-6}
600	45	1,383,454	2.09×10^{-6}	2.31×10^{-6}	1.06×10^{-7}	5.96×10^{-9}	3.23×10^{-7}
Treutler							
40	17	8,392	1.74×10^{-3}	1.25×10^{-4}	1.02×10^{-3}	2.71×10^{-4}	2.36×10^{-3}
60	21	17,896	2.58×10^{-5}	1.79×10^{-4}	6.08×10^{-5}	2.67×10^{-6}	2.63×10^{-4}
90	28	44,368	1.27×10^{-4}	6.48×10^{-5}	1.37×10^{-5}	8.96×10^{-6}	6.67×10^{-5}
130	29	64,478	1.10×10^{-4}	7.33×10^{-5}	2.16×10^{-7}	5.02×10^{-6}	3.19×10^{-5}
200	35	139,098	3.47×10^{-5}	1.55×10^{-5}	1.40×10^{-5}	2.57×10^{-6}	3.88×10^{-5}
300	41	278,260	6.31×10^{-5}	3.08×10^{-5}	9.11×10^{-6}	1.23×10^{-6}	2.44×10^{-5}
600	45	718,822	4.62×10^{-5}	1.93×10^{-5}	8.17×10^{-6}	2.00×10^{-6}	2.07×10^{-5}
Robust							
40	17	8,608	1.73×10^{-3}	1.20×10^{-4}	1.02×10^{-3}	2.72×10^{-4}	2.36×10^{-3}
60	21	19,876	1.11×10^{-4}	1.44×10^{-4}	9.86×10^{-5}	3.56×10^{-6}	1.35×10^{-4}
90	28	54,880	3.29×10^{-5}	6.60×10^{-5}	3.85×10^{-5}	7.30×10^{-6}	1.93×10^{-6}
130	29	79,526	3.60×10^{-5}	6.19×10^{-5}	1.77×10^{-5}	5.15×10^{-6}	1.34×10^{-5}
200	35	178,050	2.71×10^{-5}	3.33×10^{-5}	9.12×10^{-7}	1.45×10^{-6}	6.82×10^{-6}
300	41	366,460	8.37×10^{-6}	1.06×10^{-5}	1.42×10^{-6}	3.80×10^{-7}	1.23×10^{-6}
600	45	965,422	2.16×10^{-6}	2.41×10^{-6}	6.57×10^{-8}	2.45×10^{-9}	3.23×10^{-7}

Table 10: Per-term convergence data of the water angular-pruning study (Figures 8 and 9): absolute errors of the kinetic, nuclear-attraction, Coulomb, exchange and total unrestricted Hartree–Fock energies. Reference: uniform unpruned grid with $n_{\mathrm{rad}} = 1000$, $L = 50$ (2,917,774 points), $E_{\mathrm{kin}} = 75.9976267410$, $E_{\mathrm{ne}} = -199.0472994426$, $E_J = 46.7402184304$, $E_K = -8.9450726577$, $E_{\mathrm{scf}} = -76.0667767173$ Ha.

References

[1] J. E. Lennard-Jones. The electronic structure of some diatomic molecules. *Transactions of the Faraday Society*, 25:668, 1929.

[2] C. C. J. Roothaan. New developments in molecular orbital theory. *Reviews of Modern Physics*, 23(2):69–89, April 1951.

[3] S. F. Boys. Electronic wave functions - i. a general method of calculation for the stationary states of any molecular system. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 200(1063):542–554, February 1950.

[4] P. Hohenberg and W. Kohn. Inhomogeneous electron gas. *Physical Review*, 136(3B):B864–B871, November 1964.

[5] W. Kohn and L. J. Sham. Self-consistent equations including exchange and correlation effects. *Physical Review*, 140(4A):A1133–A1138, November 1965.

[6] A. D. Becke. A multicenter numerical integration scheme for polyatomic molecules. *The Journal of Chemical Physics*, 88(4):2547–2553, February 1988.

[7] Aron J. Cohen and Nicholas C. Handy. Density functional generalized gradient calculations using slater basis sets. *The Journal of Chemical Physics*, 117(4):1470–1478, July 2002.

[8] Mark A. Watson, Nicholas C. Handy, and Aron J. Cohen. Density functional calculations, using slater basis sets, with exact exchange. *The Journal of Chemical Physics*, 119(13):6475–6481, October 2003.

[9] E. Van Lenthe and E. J. Baerends. Optimized slater-type basis sets for the elements 1–118. *Journal of Computational Chemistry*, 24(9):1142–1156, May 2003.

[10] Mirko Franchini, Pierre Herman Theodoor Philipsen, and Lucas Visscher. The becke fuzzy cells integration scheme in the amsterdam density functional program suite. *Journal of Computational Chemistry*, 34(21):1819–1827, May 2013.

[11] S. Obara and A. Saika. Efficient recursive computation of molecular integrals over cartesian gaussian functions. *The Journal of Chemical Physics*, 84(7):3963–3974, April 1986.

[12] Reinhart Ahlrichs. A simple algebraic derivation of the obara–saika scheme for general two-electron interaction potentials. *Phys. Chem. Chem. Phys.*, 8(26):3072–3077, 2006.

[13] Vikram Gavini, Stefano Baroni, Volker Blum, David R Bowler, Alexander Buccheri, James R Chelikowsky, Sambit Das, William Dawson, Pietro Delugas, Mehmet Dogan, Claudia Draxl, Giulia Galli, Luigi Genovese, Paolo Giannozzi, Matteo Giantomassi, Xavier Gonze, Marco Govoni, François Gygi, Andris Gulans, John M Herbert, Sebastian Kokott, Thomas D Kühne, Kai-Hsin Liou, Tsuyoshi Miyazaki, Phani Motamarri, Ayako Nakata, John E Pask, Christian Plessl, Laura E Ratcliff, Ryan M Richard, Mariana Rossi, Robert Schade, Matthias Scheffler, Ole Schütt, Phanish Suryanarayana, Marc Torrent, Lionel Truflandier, Theresa L Windus, Qimen Xu, Victor W-Z Yu, and D Perez. Roadmap on electronic structure codes in the exascale era. *Modelling and Simulation in Materials Science and Engineering*, 31(6):063301, August 2023.

[14] Jorge L. Galvez Vallejo, Calum Snowdon, Ryan Stocks, Fazeleh Kazemian, Fiona Chuo Yan Yu, Christopher Seidl, Zoe Seeger, Melisa Alkan, David Poole, Bryce M. Westheimer, Mehaboob Basha, Marco De La Pierre, Alistair Rendell, Ekaterina I. Izgorodina, Mark S. Gordon, and Giuseppe M. J. Barca. Toward an extreme-scale electronic structure system. *The Journal of Chemical Physics*, 159(4), July 2023.

[15] Ryan Stocks and Giuseppe M. J. Barca. Efficient algorithms for gpu accelerated evaluation of the dft exchange-correlation functional. *Journal of Chemical Theory and Computation*, 21(20):10263–10280, October 2025.

[16] David B. Williams-Young, Abhishek Bagusetty, Wibe A. de Jong, Douglas Doerfler, Hubertus J.J. van Dam, Álvaro Vázquez-Mayagoitia, Theresa L. Windus, and Chao Yang. Achieving performance portability in gaussian basis set density functional theory on accelerator based architectures in nwchemex. *Parallel Computing*, 108:102829, December 2021.

[17] Susi Lehtola. A call to arms: Making the case for more reusable libraries. *The Journal of Chemical Physics*, 159(18):180901, November 2023.

[18] Amir Karton, Shauli Daon, and Jan M. L. Martin. W4-11: A high-confidence benchmark dataset for computational thermochemistry derived from first-principles w4 data. *Chemical Physics Letters*, 510(4–6):165–178, 2011.

[19] G. G. Hall. The molecular orbital theory of chemical valency viii. a method of calculating ionization potentials. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 205(1083):541–552, March 1951.

[20] Delano P. Chong, Erik Van Lenthe, Stan Van Gisbergen, and Evert Jan Baerends. Even-tempered slater-type orbitals revisited: From hydrogen to krypton. *Journal of Computational Chemistry*, 25(8):1030–1036, March 2004.

[21] D.P. Chong *. Augmenting basis set for time-dependent density functional theory calculation of excitation energies: Slater-type orbitals for hydrogen to krypton. *Molecular Physics*, 103(6–8):749–761, March 2005.

[22] J. C. Slater. Atomic shielding constants. *Physical Review*, 36(1):57–64, July 1930.

[23] Ernst Joachim Weniger. The strange history of B functions or how theoretical chemists and mathematicians do (not) interact. *International Journal of Quantum Chemistry*, 109(8):1706–1716, 2009.

[24] J. Fernández Rico, R. López, A. Aguado, I. Ema, and G. Ramírez. Reference program for molecular calculations with slater-type orbitals. *Journal of Computational Chemistry*, 19(11):1284–1293, August 1998.

[25] J. Grant Hill. Gaussian basis sets for molecular applications. *International Journal of Quantum Chemistry*, 113(1):21–34, October 2012.

[26] Tosio Kato. On the eigenfunctions of many-particle systems in quantum mechanics. *Communications on Pure and Applied Mathematics*, 10(2):151–177, January 1957.

[27] R. Ditchfield, W. J. Hehre, and J. A. Pople. Self-consistent molecular-orbital methods. ix. an extended gaussian-type basis for molecular-orbital studies of organic molecules. *The Journal of Chemical Physics*, 54(2):724–728, January 1971.

[28] Thom H. Dunning. Gaussian basis sets for use in correlated molecular calculations. i. the atoms boron through neon and hydrogen. *The Journal of Chemical Physics*, 90(2):1007–1023, January 1989.

[29] Susi Lehtola. Fully numerical hartree-fock and density functional calculations. i. atoms. *International Journal of Quantum Chemistry*, 119(19), April 2019.

[30] Susi Lehtola. A review on non-relativistic, fully numerical electronic structure calculations on atoms and diatomic molecules. *International Journal of Quantum Chemistry*, 119(19), May 2019.

[31] Volker Blum, Ralf Gehrke, Felix Hanke, Paula Havu, Ville Havu, Xinguo Ren, Karsten Reuter, and Matthias Scheffler. Ab initio molecular simulations with numeric atom-centered orbitals. *Computer Physics Communications*, 180(11):2175–2196, November 2009.

[32] Susi Lehtola. Fully numerical hartree-fock and density functional calculations. ii. diatomic molecules. *International Journal of Quantum Chemistry*, 119(19), April 2019.

[33] Susi Lehtola. Atomic electronic structure calculations with hermite interpolating polynomials. *The Journal of Physical Chemistry A*, 127(18):4180–4193, April 2023.

[34] Henryk Laqua, Jörg Kussmann, and Christian Ochsenfeld. An improved molecular partitioning scheme for numerical quadratures in density functional theory. *The Journal of Chemical Physics*, 149(20), November 2018.

[35] V.I. Lebedev. Quadratures on a sphere. *USSR Computational Mathematics and Mathematical Physics*, 16(2):10–24, January 1976.

[36] V. I. Lebedev. Spherical quadrature formulas exact to orders 25–29. *Siberian Mathematical Journal*, 18(1):99–107, January 1977.

[37] Oliver Treutler and Reinhart Ahlrichs. Efficient molecular numerical integration schemes. *The Journal of Chemical Physics*, 102(1):346–354, January 1995.

[38] Peter M.W Gill, Benny G Johnson, and John A Pople. A standard grid for density functional calculations. *Chemical Physics Letters*, 209(5–6):506–512, July 1993.

[39] Christopher W. Murray, Nicholas C. Handy, and Gregory J. Laming. Quadrature schemes for integrals of density functional theory. *Molecular Physics*, 78(4):997–1014, March 1993.

[40] A. Goursot, I. Pápai, and C. A. Daul. Numerical grids for density functional calculations of molecular properties. *International Journal of Quantum Chemistry*, 52(4):799–807, November 1994.

[41] Benny G. Johnson, Peter M.W. Gill, and John A. Pople. A rotationally invariant procedure for density functional calculations. *Chemical Physics Letters*, 220(6):377–384, August 1994.

[42] Michael E. Mura and Peter J. Knowles. Improved radial grids for quadrature in molecular density-functional calculations. *The Journal of Chemical Physics*, 104(24):9848–9858, June 1996.

[43] Matthias Krack and Andreas M. Köster. An adaptive numerical integrator for molecular integrals and potentials. *The Journal of Chemical Physics*, 108(8):3226–3234, February 1998.

[44] Peter M. W. Gill and Siu-Hung Chien. Radial quadrature for multiexponential integrands. *Journal of Computational Chemistry*, 24(6):732–740, March 2003.

[45] Andreas M. Köster, Roberto Flores-Moreno, and J. Ulises Reveles. Efficient and reliable numerical integration of exchange-correlation energies and potentials. *The Journal of Chemical Physics*, 121(2):681–690, June 2004.

[46] Benny G. Johnson and Michael J. Frisch. Analytic second derivatives of the gradient-corrected density functional energy. effect of quadrature weight derivatives. *Chemical Physics Letters*, 216(1–2):133–140, December 1993.

[47] Jon Baker, Jan Andzelm, Andrew Scheiner, and Bernard Delley. The effect of grid quality and weight derivatives in density functional calculations. *The Journal of Chemical Physics*, 101(10):8894–8902, November 1994.

[48] Brian N. Papas and Henry F. Schaefer. Concerning the precision of standard density functional programs: Gaussian, molpro, nwchem, q-chem, and gamess. *Journal of Molecular Structure: THEOCHEM*, 768(1–3):175–181, August 2006.

[49] David Williams-Young. Gauxc v0.1.0, 2020.

[50] H. Carter Edwards, Christian R. Trott, and Daniel Sunderland. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *Journal of Parallel and Distributed Computing*, 74(12):3202–3216, December 2014.

[51] Christian R. Trott, Damien Lebrun-Grandie, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, Rahulkumar Gayatri, Evan Harvey, Daisy S. Hollman, Dan Ibanez, Nevin Liber, Jonathan Madsen, Jeff Miles, David Poliakoff, Amy Powell, Sivasankaran Rajamanickam, Mikael Simberg, Dan Sunderland, Bruno Turcksin, and Jeremiah Wilke. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, April 2022.

[52] Sivasankaran Rajamanickam, Seher Acer, Luc Berger-Vergiat, Vinh Dang, Nathan Ellingwood, Evan Harvey, Brian Kelley, Christian R. Trott, Jeremiah Wilke, and Ichitaro Yamazaki. Kokkos kernels: Performance portable sparse/dense linear algebra and graph kernels. arXiv preprint arXiv:2103.11991, March 2021.

[53] Simon D. Hammond, Christian R. Trott, Daniel Ibanez, and Daniel Sunderland. *Profiling and Debugging Support for the Kokkos Programming Model*, pages 743–754. Springer International Publishing, 2018.

[54] David B. Williams-Young. IntegratorXX: Numerical quadratures for molecular electronic structure theory. https://github.com/wavefunction91/IntegratorXX. Accessed: 2026-07-27.

[55] Susi Lehtola and Lori A. Burns. Openorbitaloptimizer – a reusable open source library for self-consistent field calculations. *The Journal of Physical Chemistry A*, 129(25):5651–5664, June 2025.

[56] D. Lebrun-Grandié, A. Prokopenko, B. Turcksin, and S. R. Slattery. Arborx: A performance portable geometric search library. *ACM Transactions on Mathematical Software*, 47(1):1–15, December 2020.

[57] Andrey Prokopenko, Daniel Arndt, Damien Lebrun-Grandié, and Bruno Turcksin. The arborx library: Version 2.0. *ACM Transactions on Mathematical Software*, 51(4):1–10, December 2025.

[58] Daniel G. A. Smith, Lori A. Burns, Andrew C. Simmonett, Robert M. Parrish, Matthew C. Schieber, Raimondas Galvelis, Peter Kraus, Holger Kruse, Roberto Di Remigio, Asem Alenaizan, Andrew M. James, Susi Lehtola, Jonathon P. Misiewicz, Maximilian Scheurer, Robert A. Shaw, Jeffrey B. Schriber, Yi Xie, Zachary L. Glick, Dominic A. Sirianni, Joseph Senan O'Brien, Jonathan M. Waldrop, Ashutosh Kumar, Edward G. Hohenstein, Benjamin P. Pritchard, Bernard R. Brooks, Henry F. Schaefer, Alexander Yu. Sokolov, Konrad Patkowski, A. Eugene DePrince, Ugur Bozkaya, Rollin A. King, Francesco A. Evangelista, Justin M. Turney, T. Daniel Crawford, and C. David Sherrill. PSI4 1.4: Open-source software for high-throughput quantum chemistry. *The Journal of Chemical Physics*, 152(18):184108, 2020.

[59] Axel D. Becke. A new mixing of hartree–fock and local density-functional theories. *The Journal of Chemical Physics*, 98(2):1372–1377, 1993.